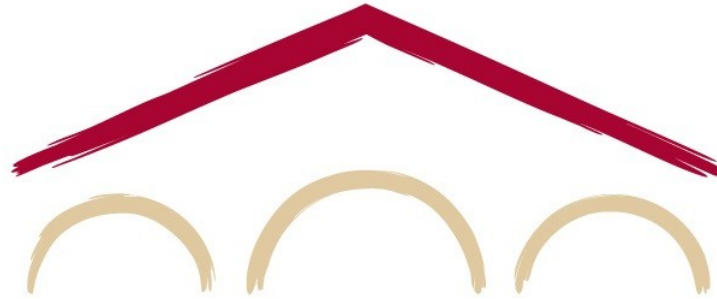


Natural Language Processing with Deep Learning

CS224N/Ling284

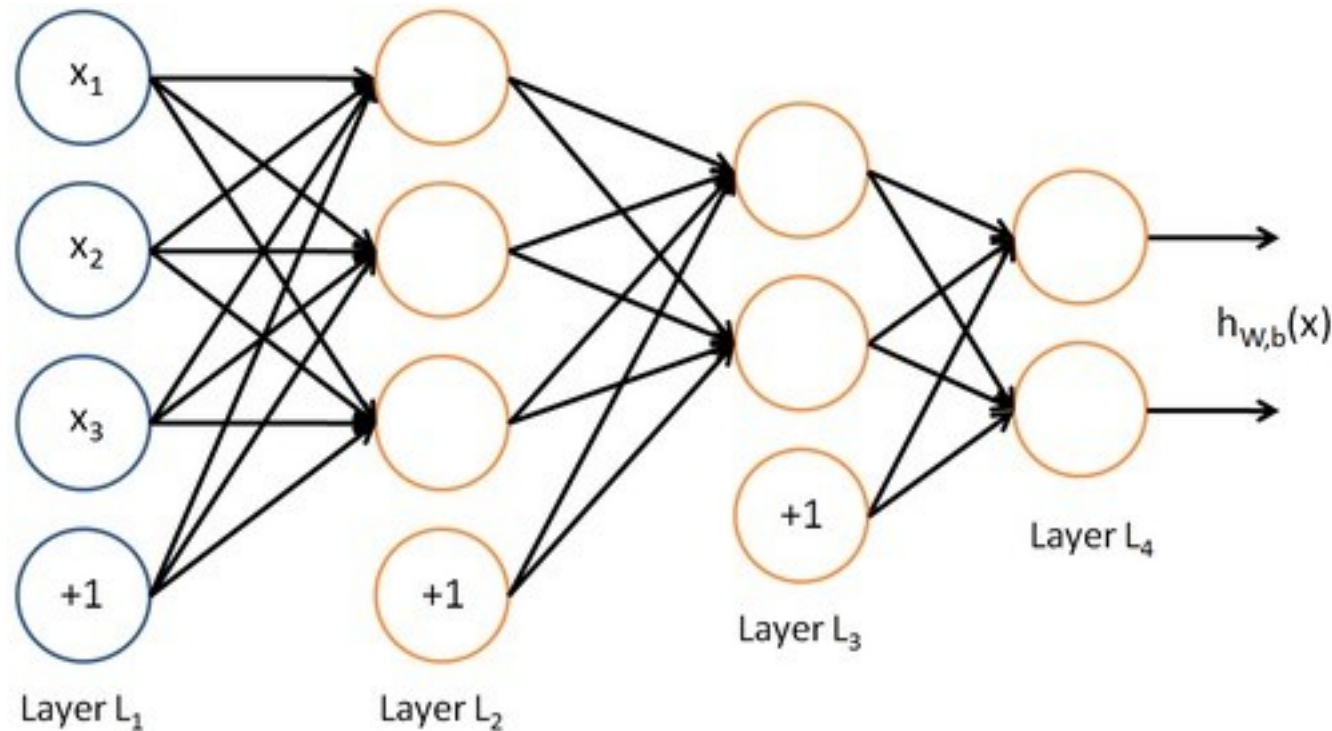


Christopher Manning

Lecture 3: Neural net learning: Gradients by hand (matrix calculus)
and algorithmically (the backpropagation algorithm)

Where we ended last time: A neural network = running several logistic regressions at the same time

Before we know it, we have a multilayer neural network....



This allows us to re-represent and compose our data multiple times and to learn a classifier that is highly non-linear in terms of the original inputs

(but, typically, is linear in terms of the pre-final layer representations)

Matrix notation for a layer

We have

$$a_1 = f(W_{11}x_1 + W_{12}x_2 + W_{13}x_3 + b_1)$$

$$a_2 = f(W_{21}x_1 + W_{22}x_2 + W_{23}x_3 + b_2)$$

etc.

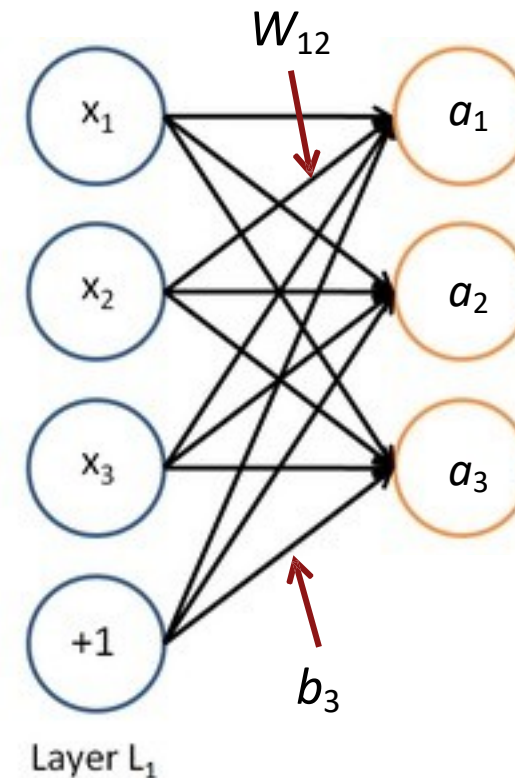
In matrix notation

$$z = Wx + b$$

$$a = f(z)$$

Activation f is applied element-wise:

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)]$$

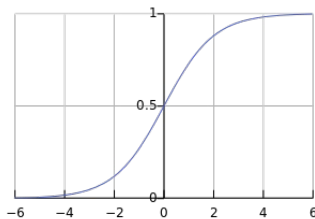


NER: Binary classification for center word being location

- We do supervised training and want high score if it's a location

$$J_t(\theta) = \sigma(s) = \frac{1}{1 + e^{-s}}$$

predicted model
probability of class



$$s = u^T h$$

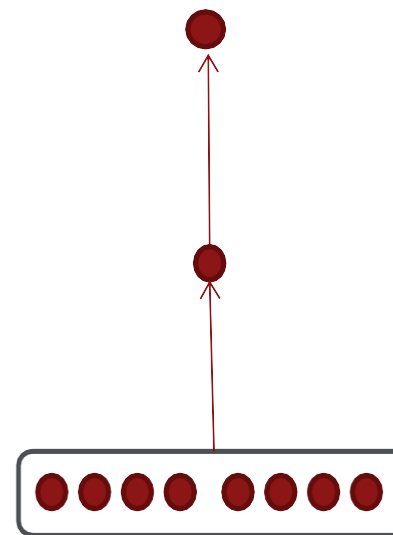
$$h = f(Wx + b)$$

$$x \text{ (input)} \in \mathbb{R}^{5d}$$



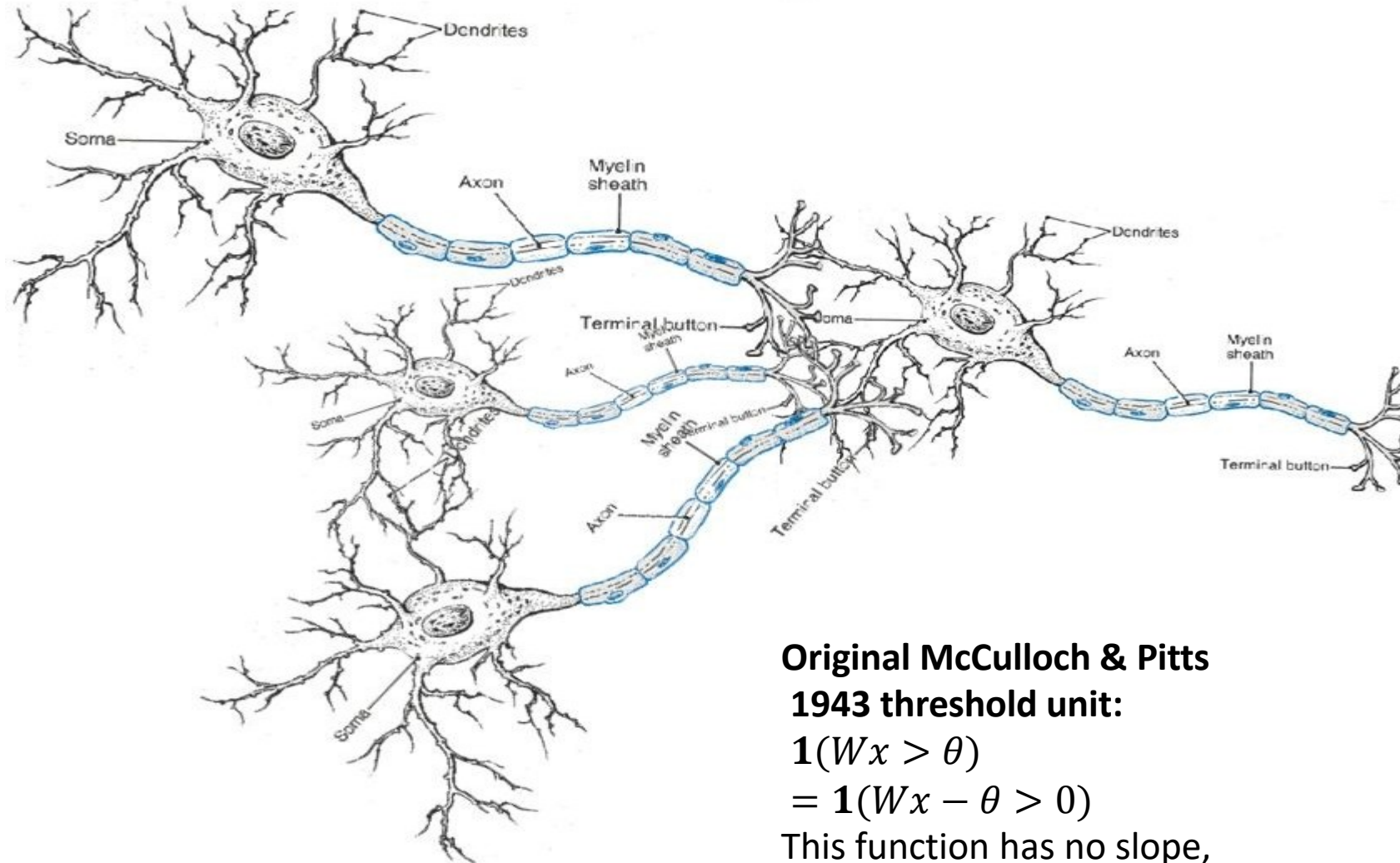
$$x = [x_{\text{museums}} \quad x_{\text{in}} \quad x_{\text{Paris}} \quad x_{\text{are}} \quad x_{\text{amazing}}]$$

f = Some element-wise non-linear function, e.g., logistic, tanh, ReLU



Embedding of
1-hot words

7. Neural computation



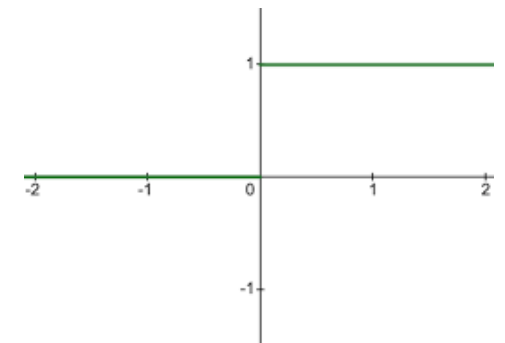
Original McCulloch & Pitts

1943 threshold unit:

$$1(Wx > \theta)$$

$$= 1(Wx - \theta > 0)$$

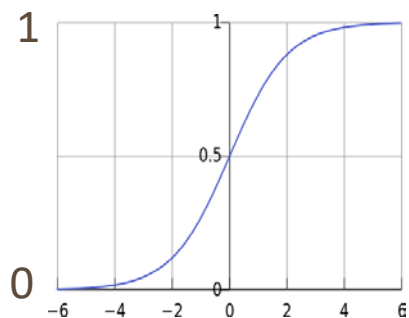
This function has no slope,
so, no **gradient-based learning**



Non-linearities, old and new

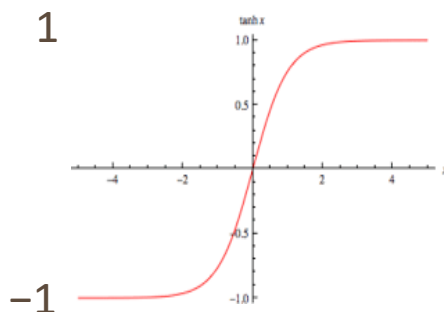
logistic (“sigmoid”)

$$f(z) = \frac{1}{1 + \exp(-z)}$$



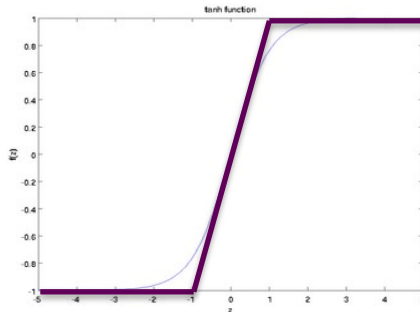
tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



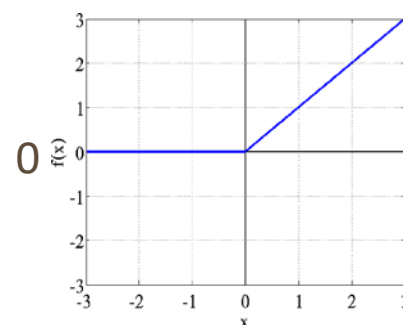
hard tanh

$$\text{HardTanh}(x) = \begin{cases} -1 & \text{if } x < -1 \\ x & \text{if } -1 \leq x \leq 1 \\ 1 & \text{if } x > 1 \end{cases}$$

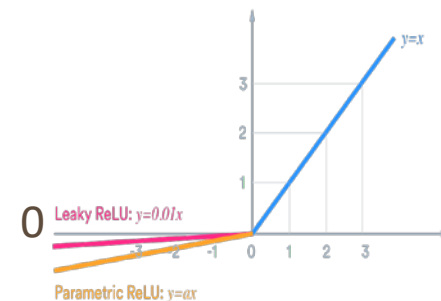


(Rectified Linear Unit)
ReLU

$$\text{ReLU}(z) = \max(z, 0)$$



Leaky ReLU /
Parametric ReLU



tanh is just a rescaled and shifted sigmoid ($2 \times$ as steep, $[-1, 1]$):

$$\tanh(z) = 2\text{logistic}(2z) - 1$$

Logistic and tanh are still used (e.g., logistic to get a probability)

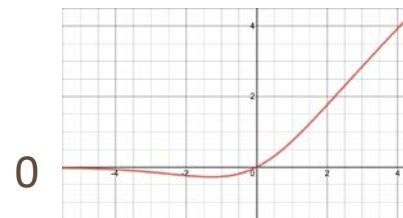
However, now, for deep networks, the first thing to try is ReLU: it trains quickly and performs well due to good gradient backflow.

ReLU has a negative “dead zone” that recent proposals mitigate

GELU/Swish often used with Transformers (BERT, RoBERTa, etc.)

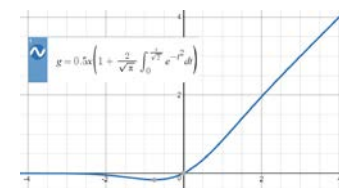
Swish [arXiv:1710.05941](https://arxiv.org/abs/1710.05941)

$$\text{swish}(x) = x \cdot \text{logistic}(x)$$



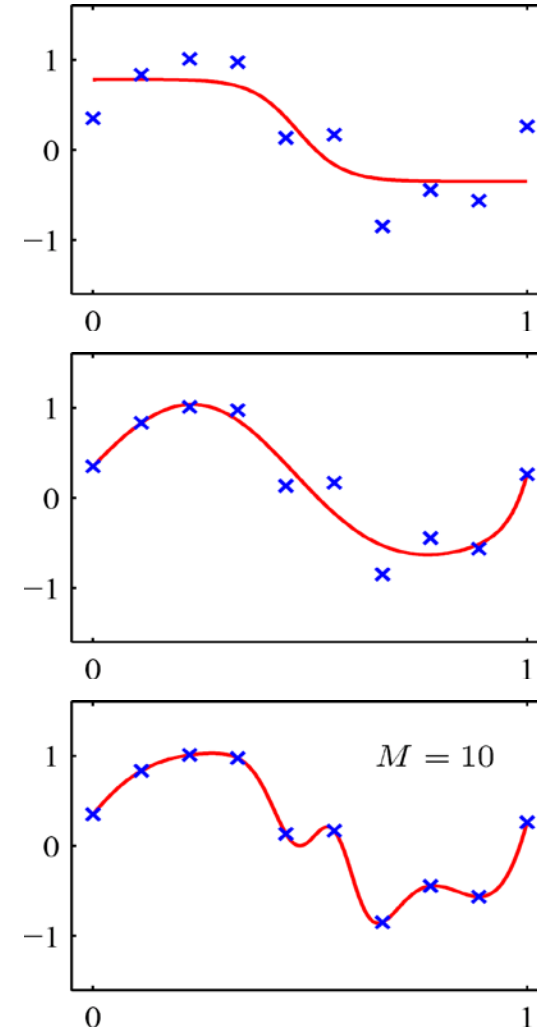
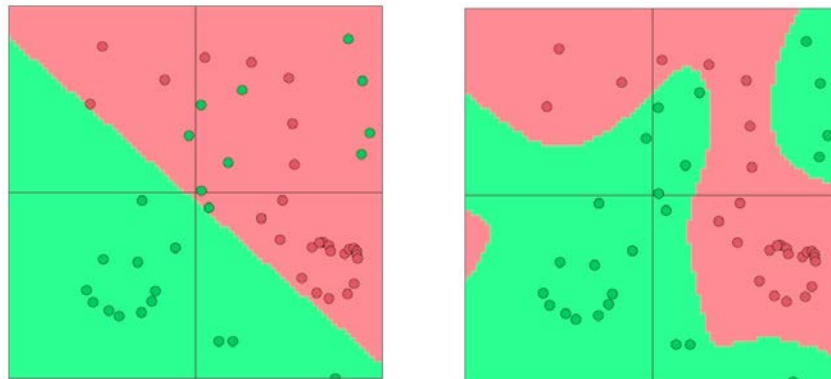
GELU [arXiv:1606.08415](https://arxiv.org/abs/1606.08415)

$$\begin{aligned} \text{GELU}(x) &= x \cdot 2P(X \leq x), X \sim N(0, 1) \\ &\approx x \cdot 2\text{logistic}(1.702x) \end{aligned}$$



Non-linearities (i.e., “ f ” on previous slide): Why they’re needed

- Neural networks do function approximation, e.g., regression or classification
 - Without non-linearities, deep neural networks can’t do anything more than a linear transform
 - Extra layers could just be compiled down into a single linear transform: $W_1 W_2 x = Wx$
 - But, with more layers that include non-linearities, they can approximate any complex function!



Remember: Stochastic Gradient Descent

Update equation:

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

$\alpha = \text{step size or learning rate}$

i.e., for each parameter: $\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial J(\theta)}{\partial \theta_j^{old}}$

In deep learning, θ includes the data representation (e.g., word vectors) too!

How can we compute $\nabla_{\theta} J(\theta)$?

1. By hand
2. Algorithmically: the backpropagation algorithm (next lecture!)

Lecture Plan

Lecture 4: Gradients by hand and algorithmically

1. Introduction (10 mins)
2. Matrix calculus (35 mins)
3. Backpropagation (35 mins)

Key Learning: The mathematics and practical implementation of how neural networks are trained by backpropagation

Linear Algebra, Multivariable Calculus, and Modern Applications

Math 51 course text prepared by the
Stanford University Math Department

Last modified on April 3, 2024

“... research on neural networks was held up for 20 years until somebody remembered the Chain Rule!”
T. Griffiths, Luce Professor of Information Technology, Consciousness, and Culture (Princeton)

“... linear algebra simplifies the complex processes that underlie neural network operations, [...] allowing [AI models] to efficiently process vast amounts of data, recognize patterns, and make decisions.”
Medium article “[The Crucial Role of Mathematics in AI Development](#)”

G. Neural networks and the multivariable Chain Rule (optional)

The ability of computers to identify high-level patterns through experience (i.e., machine-learning based on training data) has been demonstrated quite spectacularly with AlphaGo (and its descendants such as AlphaZero). This is based on the concept of a *neural network*, which we discuss here in order to identify where an essential mathematical insight occurs involving the multivariable Chain Rule.

G.1. Mathematical model of a neural network as a composition of functions. Consider the task of a computer being given a picture with 1000 pixels (encoded as a vector in $\mathbf{R}^{1000}$) and aiming to determine whether or not it is a picture of a cat. The output of the computer’s work will be a single real number between 0 and 1 that measures how likely it is that the picture is of a cat.

The overall process of feeding the picture into the computer and extracting the output from the computer should be given by evaluating a function $f_{\text{cat}} : \mathbf{R}^{1000} \rightarrow \mathbf{R}$ on a 1000-vector that encodes the input of the pixel intensities. The desired features of f_{cat} are:

- if the input is a picture of a cat, f_{cat} returns a value > 0.5 ,
- if the input is not a picture of a cat then f_{cat} returns a value < 0.5 .

This is a “classifier” problem (cat or not cat?), and the computer needs to *construct* such a function! The process of building f_{cat} inside a computer (in a way we describe below) is an instance of “machine learning”. It must be kept in mind that the function f_{cat} constructed by the computer will be an extraordinarily non-linear function, far too complicated for any human to ever discover or write down.

Two different computers working on this same “machine learning” task may build very different functions f_{cat} . All that matters is that the function does a good job at answering “cat or not cat?”; many different functions may perform equally well at this task, and we only care that the computer constructs an f_{cat} that works well in practice. The process by which f_{cat} is built inside a computer involves expressing it as a composition of many intermediate vector-valued functions (that also have to be constructed by the computer), and this all makes essential use of multivariable differential calculus, as we will explain below.

From a mathematical point of view, a *neural network* is an expression of a function $f : \mathbf{R}^n \rightarrow \mathbf{R}^m$ (often $m = 1$, as for f_{cat} above) as a composition of (typically highly non-linear) vector-valued functions:

$$\mathbf{R}^n \xrightarrow{f_1} \mathbf{R}^{d_1} \xrightarrow{f_2} \mathbf{R}^{d_2} \xrightarrow{f_3} \cdots \xrightarrow{f_{N-1}} \mathbf{R}^{d_{N-1}} \xrightarrow{f_N} \mathbf{R}^m$$

in which $\mathbf{R}^n$ on the left is called the *input layer*, $\mathbf{R}^m$ on the right is called the *output layer*, and each $\mathbf{R}^{d_i}$ in between is called a *hidden layer*. Such a composition involving N functions f_i is called an N -layer neural network (so a single-layer network involves no hidden layers and so no function composition at all).

Defining $d_0 = n$ and $d_N = m$ for convenience of notation, we refer to $\mathbf{R}^{d_j}$ as the j th *layer* (with $j = 0, 1, \dots, N$) and call d_j the number of *nodes* in that layer. Each function f_i describes how to get from the output of the $(i - 1)$ th layer (which means just the input when $i = 1$) to the next layer. If we denote a vector in $\mathbf{R}^{d_j}$ as $\mathbf{x}_j = (x_{1,j}, \dots, x_{d_j,j})$ then (for reasons based on analogies with the behavior of neurons in a brain) each $x_{i,j}$ is called a *neuron*, hence the name “neural network” for the overall function composition process. Many people dislike the analogies with a brain and so avoid such terminology, instead referring to each $x_{i,j}$ as a *unit* and calling the overall composition process a *multi-layer perceptron*.

Computing Gradients by Hand

- **Matrix calculus:** Fully vectorized gradients
 - “Multivariable calculus is just like single-variable calculus if you use matrices”
 - Much faster and more useful than non-vectorized gradients
 - But doing a non-vectorized gradient can be good for intuition; recall the first lecture for an example
 - **Lecture notes and matrix calculus notes cover this material in more detail**
 - **You might also review Math 51, which has an online textbook:**
<http://web.stanford.edu/class/math51/textbook.html>

Gradients

- Given a function with 1 output and 1 input

$$f(x) = x^3$$

- It's gradient (slope) is its derivative

$$\frac{df}{dx} = 3x^2$$

“How much will the output change if we change the input a bit?”

At $x = 1$ it changes about 3 times as much: $1.01^3 = 1.03$

At $x = 4$ it changes about 48 times as much: $4.01^3 = 64.48$

Gradients

- Given a function with 1 output and n inputs

$$f(\mathbf{x}) = f(x_1, x_2, \dots, x_n)$$

- Its gradient is a vector of partial derivatives with respect to each input

$$\frac{\partial f}{\partial \mathbf{x}} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Jacobian Matrix: Generalization of the Gradient

- Given a function with **m outputs** and n inputs

$$\mathbf{f}(\mathbf{x}) = [f_1(x_1, x_2, \dots, x_n), \dots, f_m(x_1, x_2, \dots, x_n)]$$

- It's Jacobian is an **$m \times n$ matrix** of partial derivatives

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

$$\left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right)_{ij} = \frac{\partial f_i}{\partial x_j}$$

Chain Rule

- For composition of one-variable functions: **multiply derivatives**

$$z = 3y$$

$$y = x^2$$

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} = (3)(2x) = 6x$$

- For multiple variables functions: **multiply Jacobians**

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \dots$$

Example Jacobian: Elementwise activation Function

$$\mathbf{h} = f(\mathbf{z}), \text{ what is } \frac{\partial \mathbf{h}}{\partial \mathbf{z}}? \quad \mathbf{h}, \mathbf{z} \in \mathbb{R}^n$$
$$h_i = f(z_i)$$

Example Jacobian: Elementwise activation Function

$$\mathbf{h} = f(\mathbf{z}), \text{ what is } \frac{\partial \mathbf{h}}{\partial \mathbf{z}}? \quad \mathbf{h}, \mathbf{z} \in \mathbb{R}^n$$
$$h_i = f(z_i)$$

Function has n outputs and n inputs $\rightarrow n$ by n Jacobian

Example Jacobian: Elementwise activation Function

$$\mathbf{h} = f(\mathbf{z}), \text{ what is } \frac{\partial \mathbf{h}}{\partial \mathbf{z}}?$$

$$h_i = f(z_i)$$

$$\mathbf{h}, \mathbf{z} \in \mathbb{R}^n$$

$$\left(\frac{\partial \mathbf{h}}{\partial \mathbf{z}} \right)_{ij} = \frac{\partial h_i}{\partial z_j} = \frac{\partial}{\partial z_j} f(z_i)$$

definition of Jacobian

Example Jacobian: Elementwise activation Function

$$\mathbf{h} = f(\mathbf{z}), \text{ what is } \frac{\partial \mathbf{h}}{\partial \mathbf{z}}?$$

$$h_i = f(z_i)$$

$$\mathbf{h}, \mathbf{z} \in \mathbb{R}^n$$

$$\left(\frac{\partial \mathbf{h}}{\partial \mathbf{z}} \right)_{ij} = \frac{\partial h_i}{\partial z_j} = \frac{\partial}{\partial z_j} f(z_i)$$

definition of Jacobian

$$= \begin{cases} f'(z_i) & \text{if } i = j \\ 0 & \text{if otherwise} \end{cases}$$

regular 1-variable derivative

Example Jacobian: Elementwise activation Function

$$\mathbf{h} = f(\mathbf{z}), \text{ what is } \frac{\partial \mathbf{h}}{\partial \mathbf{z}}?$$

$$\mathbf{h}, \mathbf{z} \in \mathbb{R}^n$$

$$h_i = f(z_i)$$

$$\left(\frac{\partial \mathbf{h}}{\partial \mathbf{z}} \right)_{ij} = \frac{\partial h_i}{\partial z_j} = \frac{\partial}{\partial z_j} f(z_i)$$

definition of Jacobian

$$= \begin{cases} f'(z_i) & \text{if } i = j \\ 0 & \text{if otherwise} \end{cases}$$

regular 1-variable derivative

$$\frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \begin{pmatrix} f'(z_1) & & 0 \\ & \ddots & \\ 0 & & f'(z_n) \end{pmatrix} = \text{diag}(\mathbf{f}'(\mathbf{z}))$$

Other Jacobians

$$\frac{\partial}{\partial x}(\mathbf{W}x + \mathbf{b}) = \mathbf{W}$$

Other Jacobians

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{W}$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I} \text{ (Identity matrix)}$$

Other Jacobians

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{W}$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I} \text{ (Identity matrix)}$$

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

Fine print: This is the correct Jacobian.
Later we discuss the “shape convention”;
using it the answer would be $\mathbf{h}$.

Other Jacobians

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{W}$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I} \text{ (Identity matrix)}$$

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

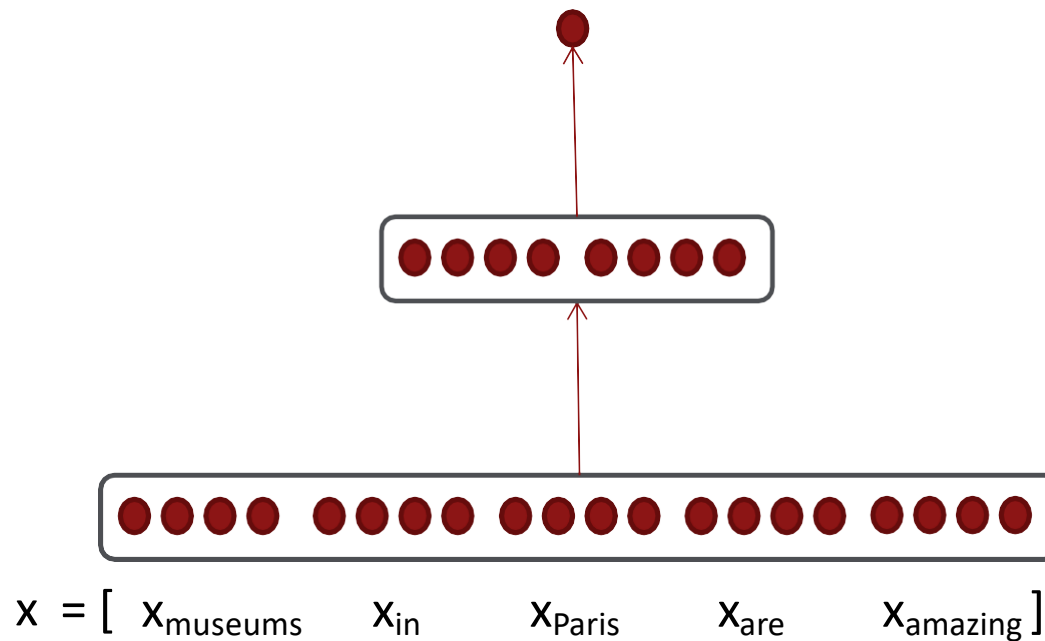
- Compute these at home for practice!
 - Check your answers with the lecture notes

Back to our Neural Net!

$$s = u^T h$$

$$h = f(Wx + b)$$

x (input)



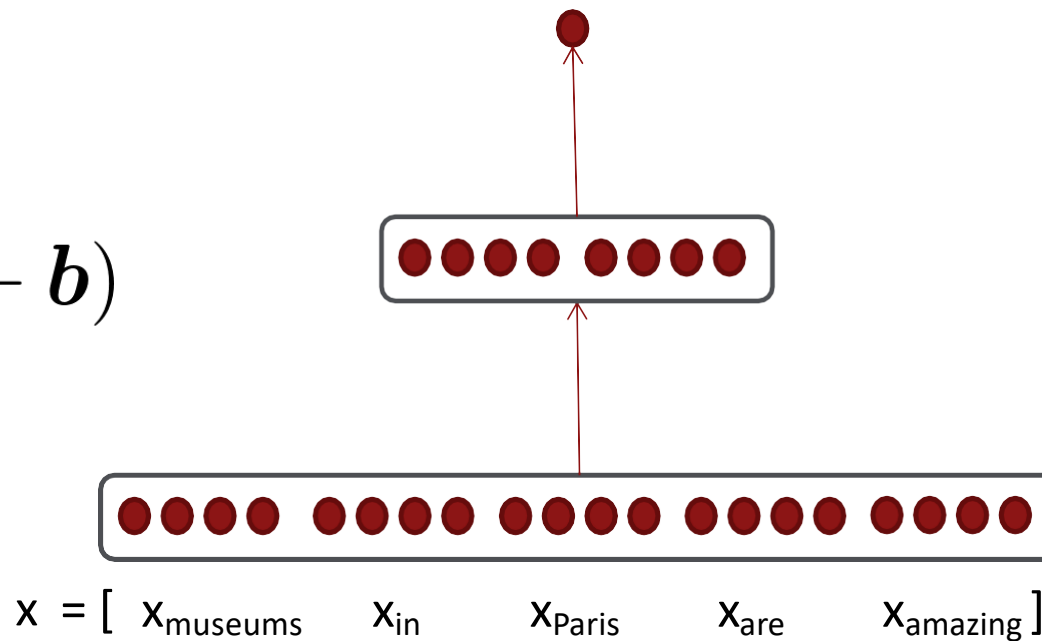
Back to our Neural Net!

- Let's find $\frac{\partial s}{\partial b}$
 - Really, we care about the gradient of the loss J_t but we will compute the gradient of the score for simplicity (and because the logistic is in Ass 1!)

$$s = u^T h$$

$$h = f(Wx + b)$$

x (input)



1. Break up equations into simple pieces

$$s = \mathbf{u}^T \mathbf{h}$$

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{W}\mathbf{x} + \mathbf{b}) \quad \rightarrow \quad \mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{x} \quad (\text{input})$$

$$\mathbf{x} \quad (\text{input})$$

Carefully define your variables and keep track of their dimensionality!

2. Apply the chain rule

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{x} \quad (\text{input})$$

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

2. Apply the chain rule

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{x} \quad (\text{input})$$

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

2. Apply the chain rule

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$\mathbf{x}$ (input)

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

2. Apply the chain rule

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$\mathbf{x}$ (input)

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

3. Write out the Jacobians

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{x} \quad (\text{input})$$

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

Useful Jacobians from previous slide

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

$$\frac{\partial}{\partial \mathbf{z}} (f(\mathbf{z})) = \text{diag}(f'(\mathbf{z}))$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I}$$

3. Write out the Jacobians

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{x} \quad (\text{input})$$

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \quad \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \quad \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

$\downarrow$

$$\mathbf{u}^T$$

Useful Jacobians from previous slide

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

$$\frac{\partial}{\partial \mathbf{z}} (f(\mathbf{z})) = \text{diag}(f'(\mathbf{z}))$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I}$$

3. Write out the Jacobians

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$\mathbf{x}$ (input)

$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \quad \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \quad \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

$\downarrow \qquad \qquad \downarrow$

$$\mathbf{u}^T \text{diag}(f'(\mathbf{z}))$$

Useful Jacobians from previous slide

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

$$\frac{\partial}{\partial \mathbf{z}} (f(\mathbf{z})) = \text{diag}(f'(\mathbf{z}))$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I}$$

3. Write out the Jacobians

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$\mathbf{x}$ (input)

$$\begin{aligned} \frac{\partial s}{\partial \mathbf{b}} &= \frac{\partial s}{\partial \mathbf{h}} \quad \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \quad \frac{\partial \mathbf{z}}{\partial \mathbf{b}} \\ &\quad \downarrow \quad \quad \downarrow \quad \quad \downarrow \\ &= \mathbf{u}^T \text{diag}(f'(\mathbf{z})) \mathbf{I} \end{aligned}$$

Useful Jacobians from previous slide

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

$$\frac{\partial}{\partial \mathbf{z}} (f(\mathbf{z})) = \text{diag}(f'(\mathbf{z}))$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I}$$

3. Write out the Jacobians

$$s = \mathbf{u}^T \mathbf{h}$$

$$\mathbf{h} = f(\mathbf{z})$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{x} \quad (\text{input})$$

$$\begin{aligned} \frac{\partial s}{\partial \mathbf{b}} &= \frac{\partial s}{\partial \mathbf{h}} \quad \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \quad \frac{\partial \mathbf{z}}{\partial \mathbf{b}} \\ &\quad \downarrow \quad \quad \downarrow \quad \quad \downarrow \\ &= \mathbf{u}^T \text{diag}(f'(\mathbf{z})) \mathbf{I} \\ &= \mathbf{u}^T \odot f'(\mathbf{z}) \end{aligned}$$

Useful Jacobians from previous slide

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{h}) = \mathbf{h}^T$$

$$\frac{\partial}{\partial \mathbf{z}} (f(\mathbf{z})) = \text{diag}(f'(\mathbf{z}))$$

$$\frac{\partial}{\partial \mathbf{b}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I}$$

$\odot$ = Hadamard product =
element-wise multiplication
of 2 vectors to give vector

Re-using Computation

- Suppose we now want to compute $\frac{\partial s}{\partial \mathbf{W}}$
 - Using the chain rule again:

$$\frac{\partial s}{\partial \mathbf{W}} = \frac{\partial s}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial \mathbf{W}}$$

Re-using Computation

- Suppose we now want to compute $\frac{\partial s}{\partial \mathbf{W}}$
 - Using the chain rule again:

$$\frac{\partial s}{\partial \mathbf{W}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$$
$$\frac{\partial s}{\partial \mathbf{b}} = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \frac{\partial \mathbf{z}}{\partial \mathbf{b}}$$

The same! Let's avoid duplicated computation ...

Re-using Computation

- Suppose we now want to compute $\frac{\partial s}{\partial \mathbf{W}}$
 - Using the chain rule again:

$$\frac{\partial s}{\partial \mathbf{W}} = \delta \frac{\partial z}{\partial \mathbf{W}}$$

$$\frac{\partial s}{\partial \mathbf{b}} = \delta \frac{\partial z}{\partial \mathbf{b}} = \delta$$

$$\delta = \frac{\partial s}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \mathbf{u}^T \circ f'(\mathbf{z})$$

δ is the upstream gradient (“error signal”)

Derivative with respect to Matrix: Output shape

- What does $\frac{\partial s}{\partial \mathbf{W}}$ look like? $\mathbf{W} \in \mathbb{R}^{n \times m}$
- 1 output, nm inputs: 1 by nm Jacobian?
 - Inconvenient to then do $\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$

Derivative with respect to Matrix: Output shape

- What does $\frac{\partial s}{\partial \mathbf{W}}$ look like? $\mathbf{W} \in \mathbb{R}^{n \times m}$
- 1 output, nm inputs: 1 by nm Jacobian?
 - Inconvenient to then do $\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$
- Instead, we leave pure math and use the **shape convention**: the shape of the gradient is the shape of the parameters!

- So $\frac{\partial s}{\partial \mathbf{W}}$ is n by m :
$$\begin{bmatrix} \frac{\partial s}{\partial W_{11}} & \cdots & \frac{\partial s}{\partial W_{1m}} \\ \vdots & \ddots & \vdots \\ \frac{\partial s}{\partial W_{n1}} & \cdots & \frac{\partial s}{\partial W_{nm}} \end{bmatrix}$$

Derivative with respect to Matrix

- What is $\frac{\partial s}{\partial \mathbf{W}} = \delta \frac{\partial z}{\partial \mathbf{W}}$
 - δ is going to be in our answer
 - The other term should be $\mathbf{x}$ because $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$

- Answer is: $\frac{\partial s}{\partial \mathbf{W}} = \delta^T \mathbf{x}^T$

δ is upstream gradient (“error signal”) at z
 $\mathbf{x}$ is local input signal

Why the Transposes?

$$\begin{aligned} \frac{\partial s}{\partial \mathbf{W}} &= \boldsymbol{\delta}^T \mathbf{x}^T \\ [n \times m] \quad [n \times 1][1 \times m] \\ &= \begin{bmatrix} \delta_1 \\ \vdots \\ \delta_n \end{bmatrix} [x_1, \dots, x_m] = \begin{bmatrix} \delta_1 x_1 & \dots & \delta_1 x_m \\ \vdots & \ddots & \vdots \\ \delta_n x_1 & \dots & \delta_n x_m \end{bmatrix} \end{aligned}$$

- Hacky answer: this makes the dimensions work out!
 - Useful trick for checking your work!
- Full explanation in the lecture notes
 - Each input goes to each output – you want to get outer product

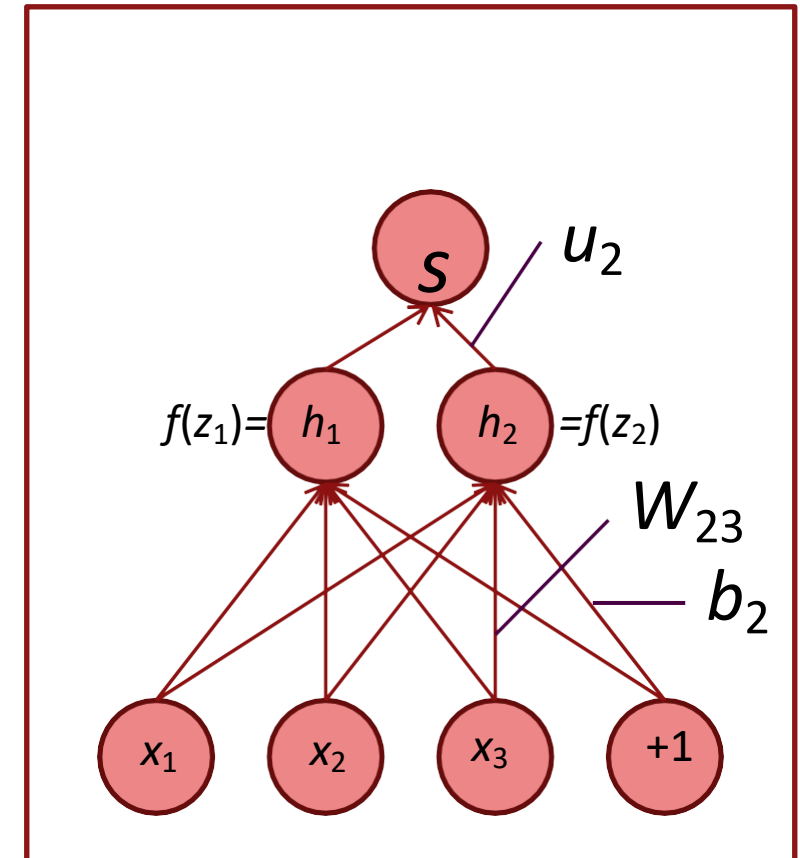
Deriving local input gradient in backprop

- For $\frac{\partial z}{\partial W}$ in our equation:

$$\frac{\partial s}{\partial W} = \delta \frac{\partial z}{\partial W} = \delta \frac{\partial}{\partial W} (Wx + b)$$

- Let's consider the derivative of a single weight W_{ij}
- W_{ij} only contributes to z_i
 - For example: W_{23} is only used to compute z_2 not z_1

$$\begin{aligned} \frac{\partial z_1}{\partial W_{1j}} &= \frac{\partial}{\partial W_{1j}} W_{12}x + \\ &= \frac{\partial}{\partial W_{1j}} \sum_k W_{1k}x_k = x_j \end{aligned}$$



What shape should derivatives be?

- Similarly, $\frac{\partial s}{\partial \mathbf{b}} = \mathbf{h}^T \circ f'(z)$ is a row vector
 - But shape convention says our gradient should be a column vector because $\mathbf{b}$ is a column vector ...
- Disagreement between Jacobian form (which makes the chain rule easy) and the shape convention (which makes implementing SGD easy)
 - We expect answers in the assignment to follow the **shape convention**
 - But Jacobian form is useful for computing the answers

What shape should derivatives be?

Two options for working through specific problems:

1. Use Jacobian form as much as possible, reshape to follow the shape convention at the end:
 - What we just did. But at the end transpose $\frac{\partial s}{\partial \mathbf{b}}$ to make the derivative a column vector, resulting in δ^T
2. Always follow the shape convention
 - Look at dimensions to figure out when to transpose and/or reorder terms
 - The error message δ that arrives at a hidden layer has the same dimensionality as that hidden layer

3. Backpropagation

We've almost shown you backpropagation

It's taking derivatives and using the (generalized, multivariate, or matrix) chain rule

One more concept:

We **re-use** derivatives computed for higher layers in computing derivatives for lower layers to minimize computation

Computation Graphs and Backpropagation

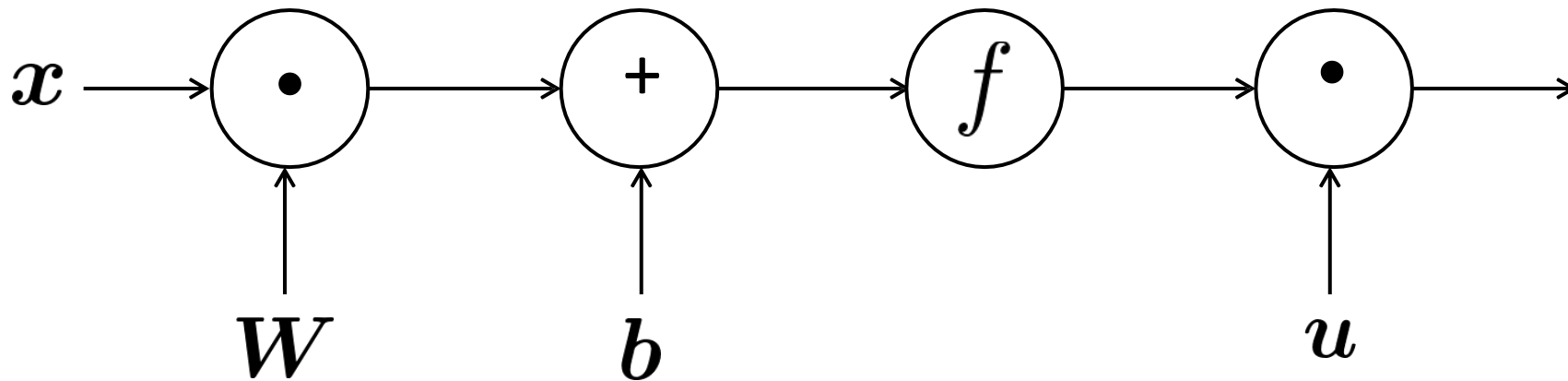
- Software represents our neural net equations as a graph
 - Source nodes: inputs
 - Interior nodes: operations

$$s = u^T h$$

$$h = f(z)$$

$$z = \mathbf{W}x + b$$

$$x \text{ (input)}$$



Computation Graphs and Backpropagation

- Software represents our neural net equations as a graph

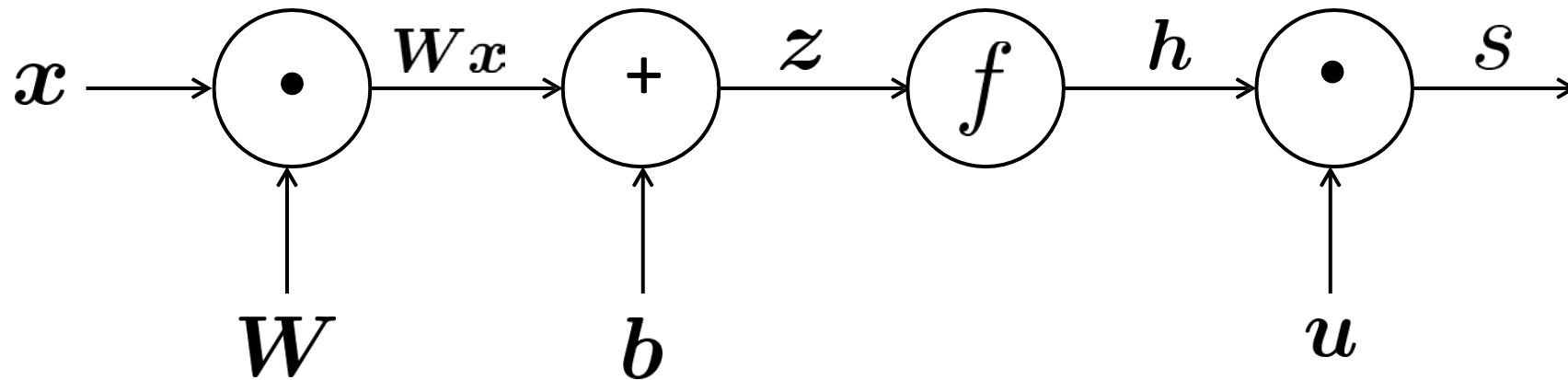
- Source nodes: inputs
- Interior nodes: operations
- Edges pass along result of the operation

$$s = u^T h$$

$$h = f(z)$$

$$z = Wx + b$$

$$x \text{ (input)}$$



Computation Graphs and Backpropagation

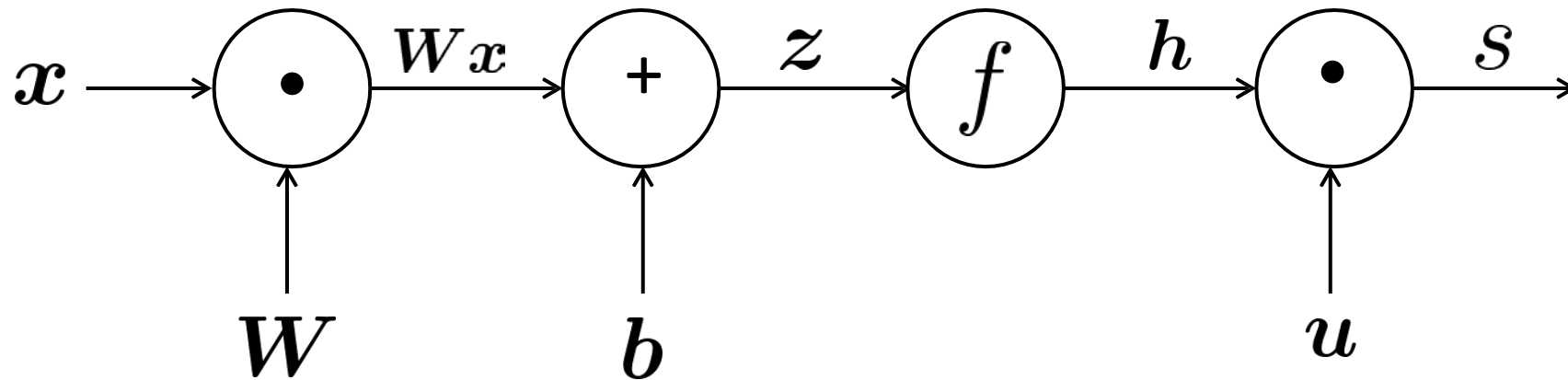
- Software represents our neural net equations as a graph

$$s = u^T h$$

$$h = f(z)$$

“Forward Propagation”

operation



Backpropagation

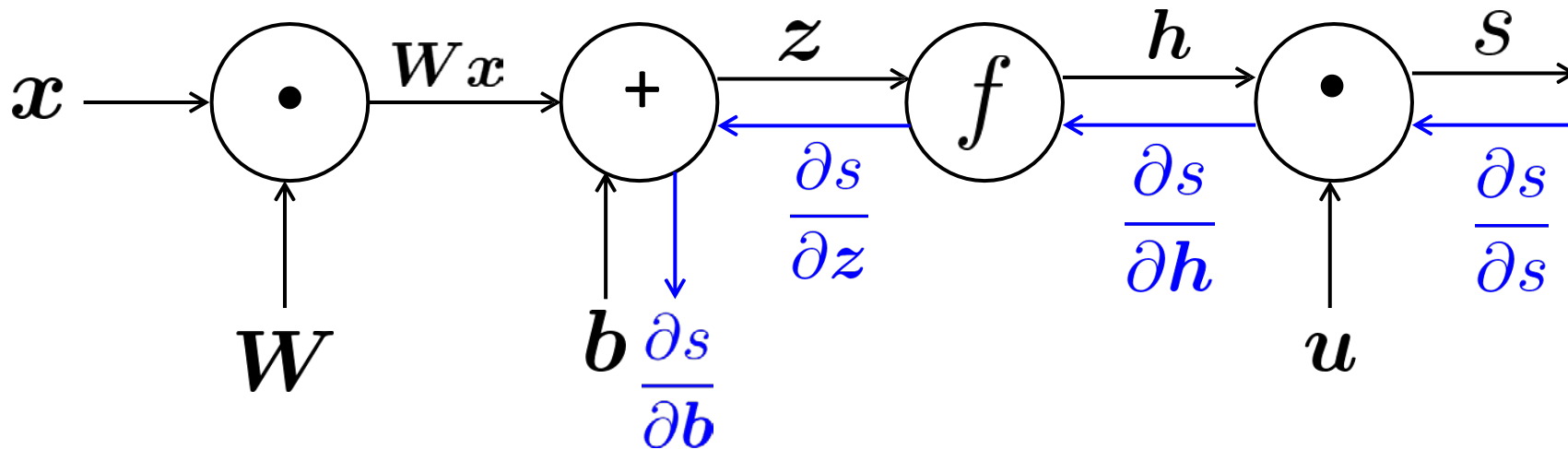
- Then go backwards along edges
 - Pass along **gradients**

$$s = u^T h$$

$$h = f(z)$$

$$z = Wx + b$$

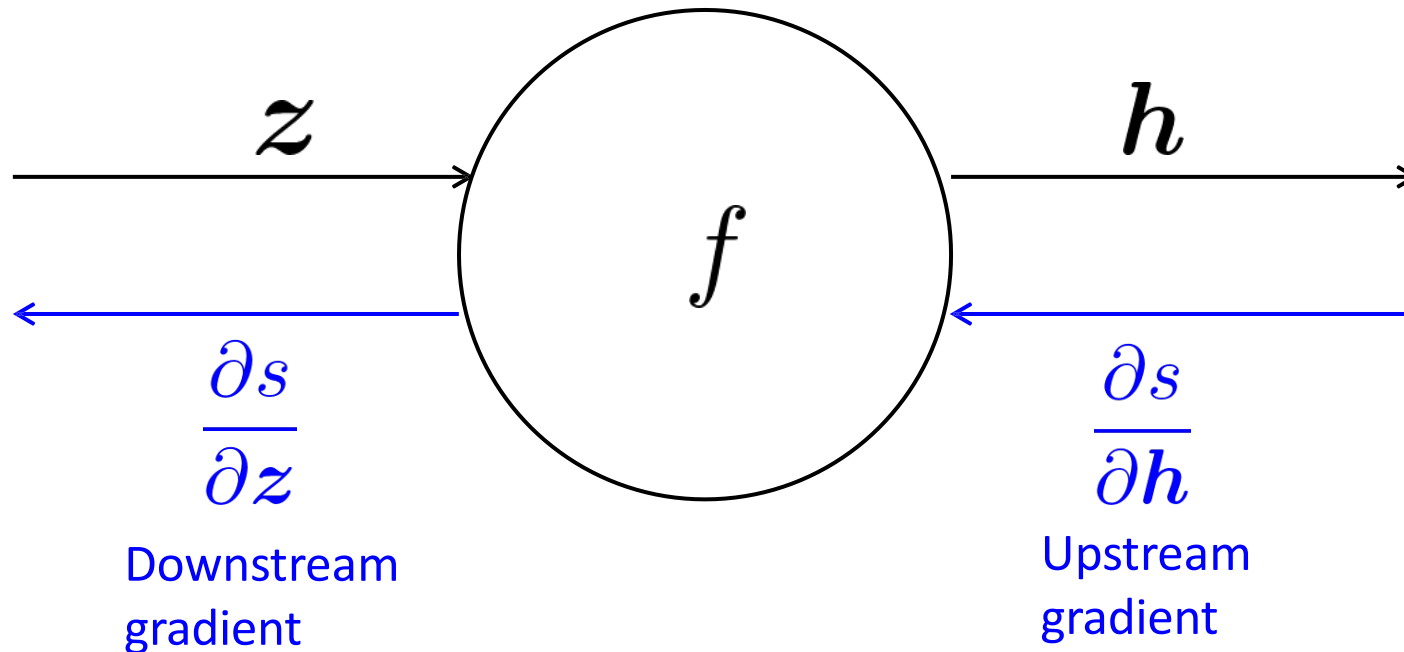
$$x \text{ (input)}$$



Backpropagation: Single Node

- Node receives an “upstream gradient”
- Goal is to pass on the correct “downstream gradient”

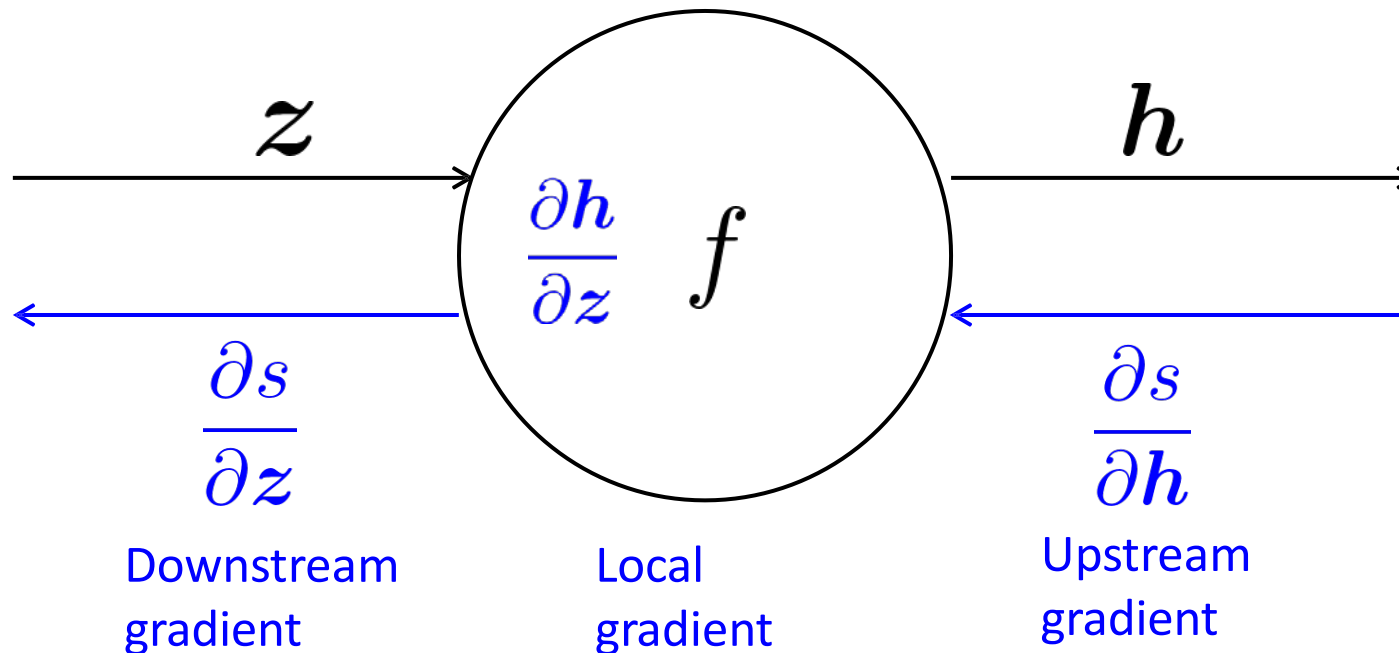
$$h = f(z)$$



Backpropagation: Single Node

- Each node has a **local gradient**
 - The gradient of its output with respect to its input

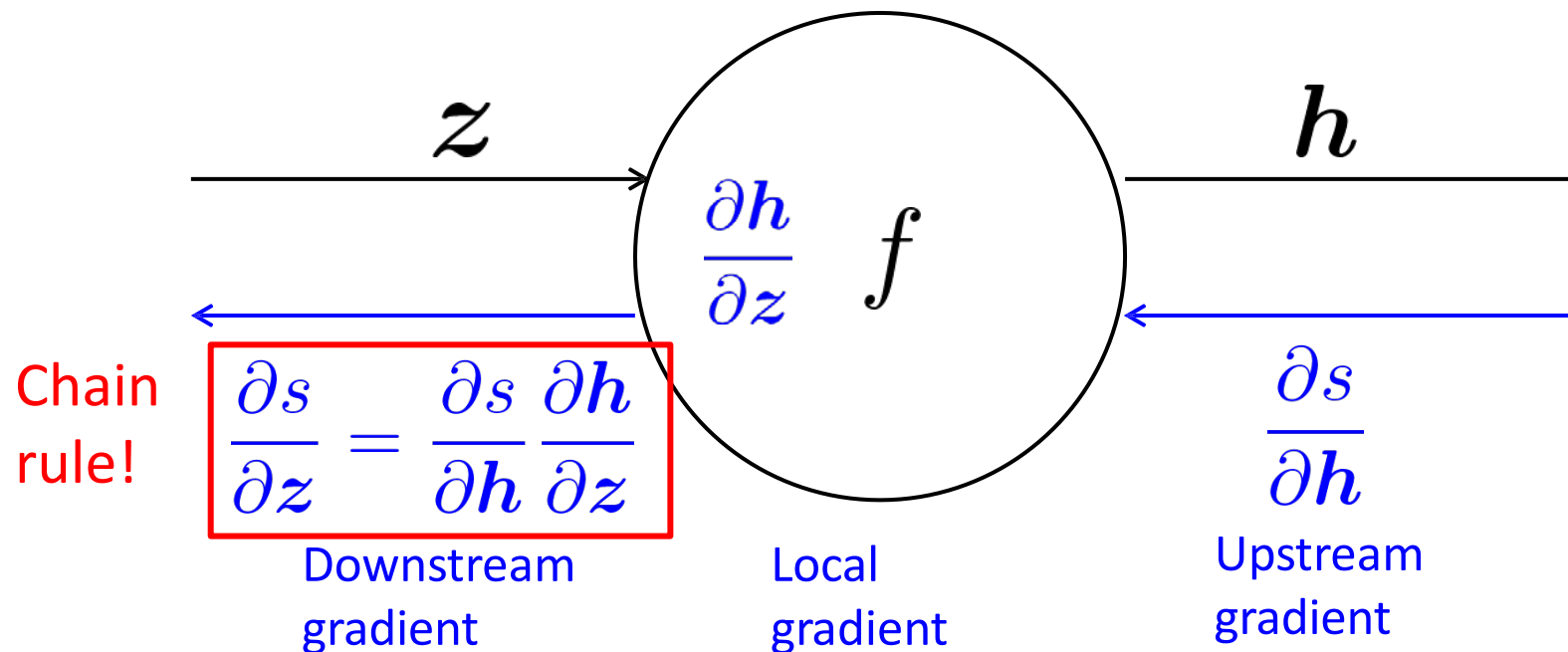
$$h = f(z)$$



Backpropagation: Single Node

- Each node has a **local gradient**
- The gradient of its output with respect to its input

$$h = f(z)$$

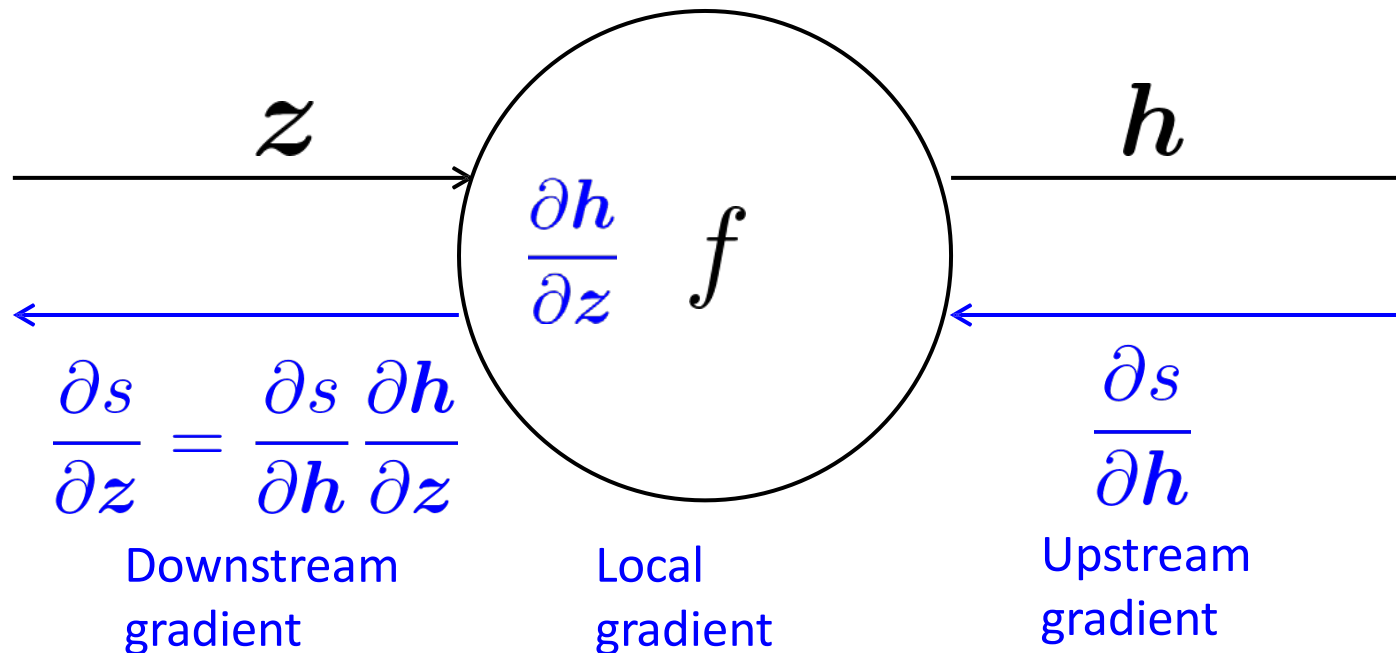


Backpropagation: Single Node

- Each node has a **local gradient**
 - The gradient of its output with respect to its input

$$h = f(z)$$

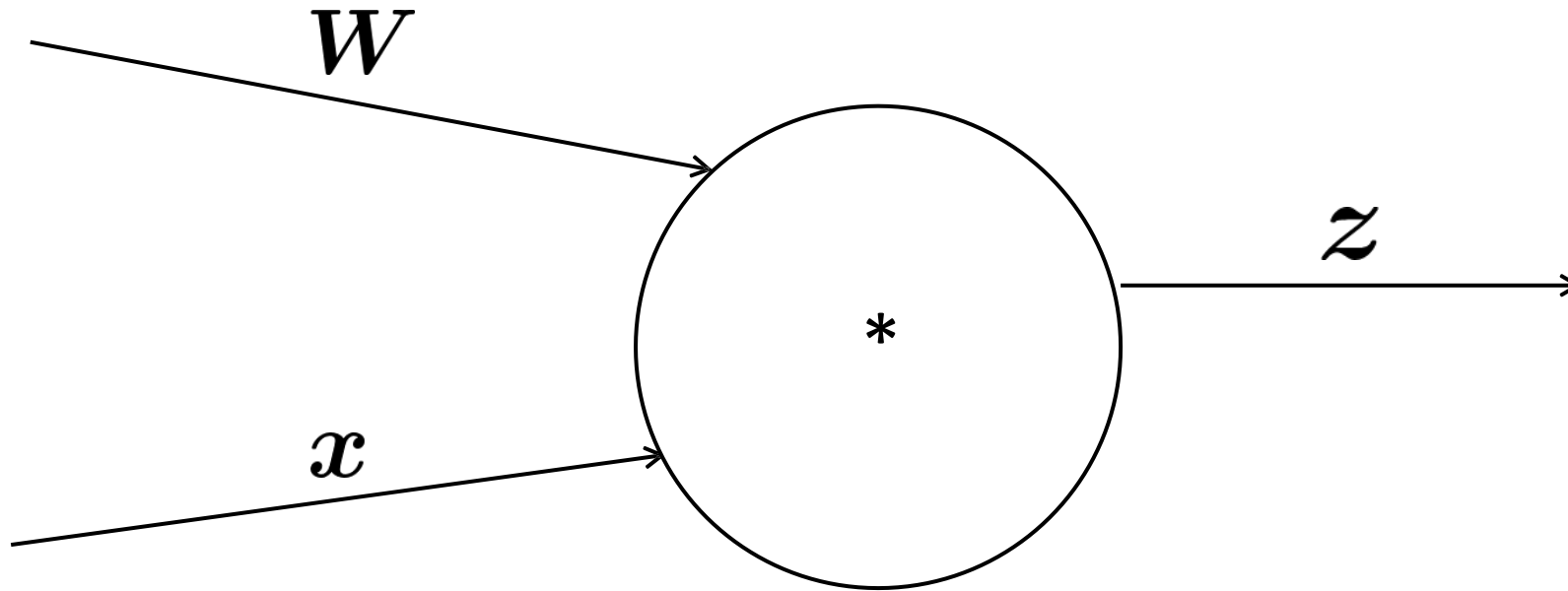
- [downstream gradient] = [upstream gradient] x [local gradient]



Backpropagation: Single Node

- What about nodes with multiple inputs?

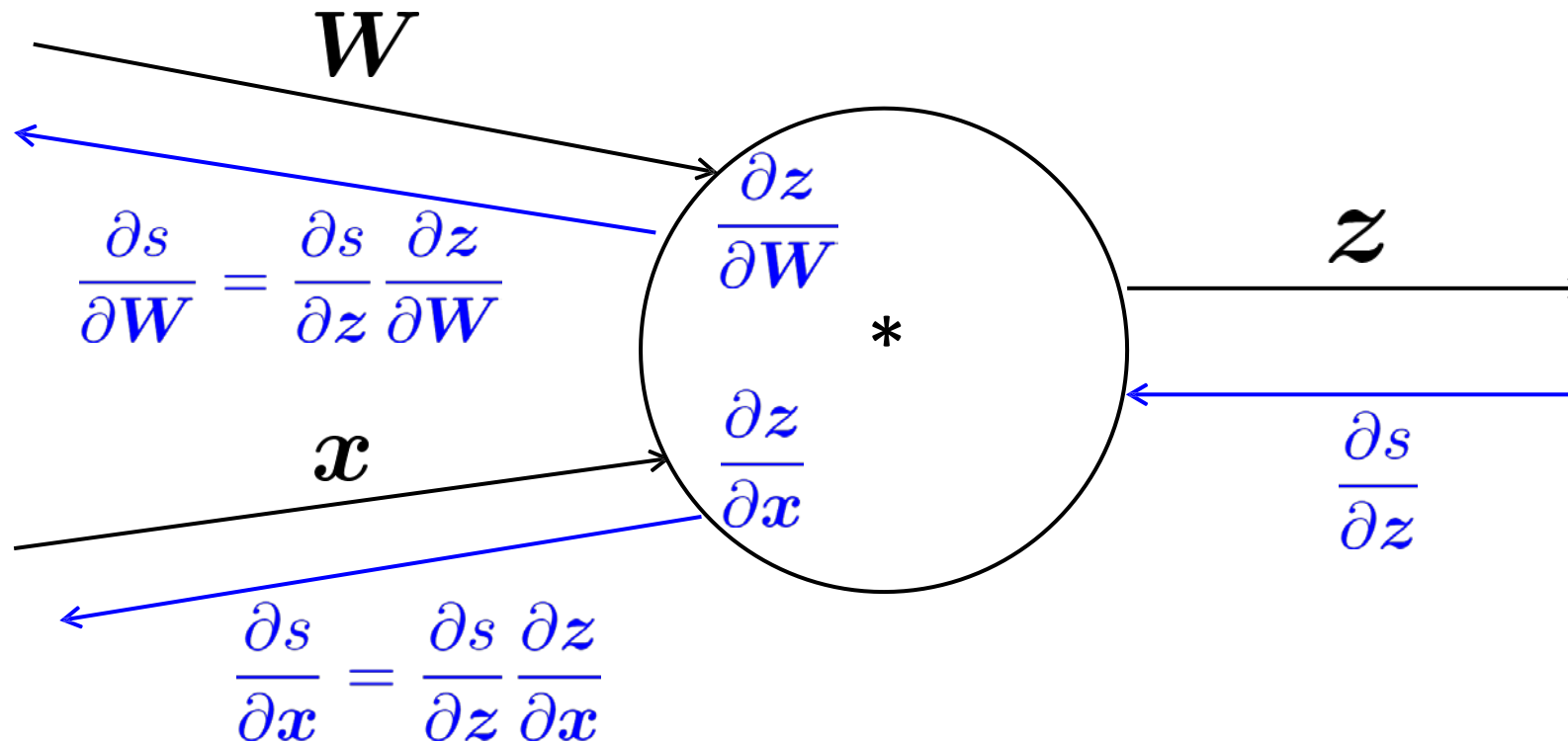
$$z = Wx$$



Backpropagation: Single Node

- Multiple inputs $\rightarrow$ multiple local gradients

$$z = Wx$$



Downstream
gradients

Local
gradients

Upstream
gradient

An Example

$$f(x, y, z) = (x + y) \max(y, z)$$

$$x = 1, y = 2, z = 0$$

An Example

$$f(x, y, z) = (x + y) \max(y, z)$$

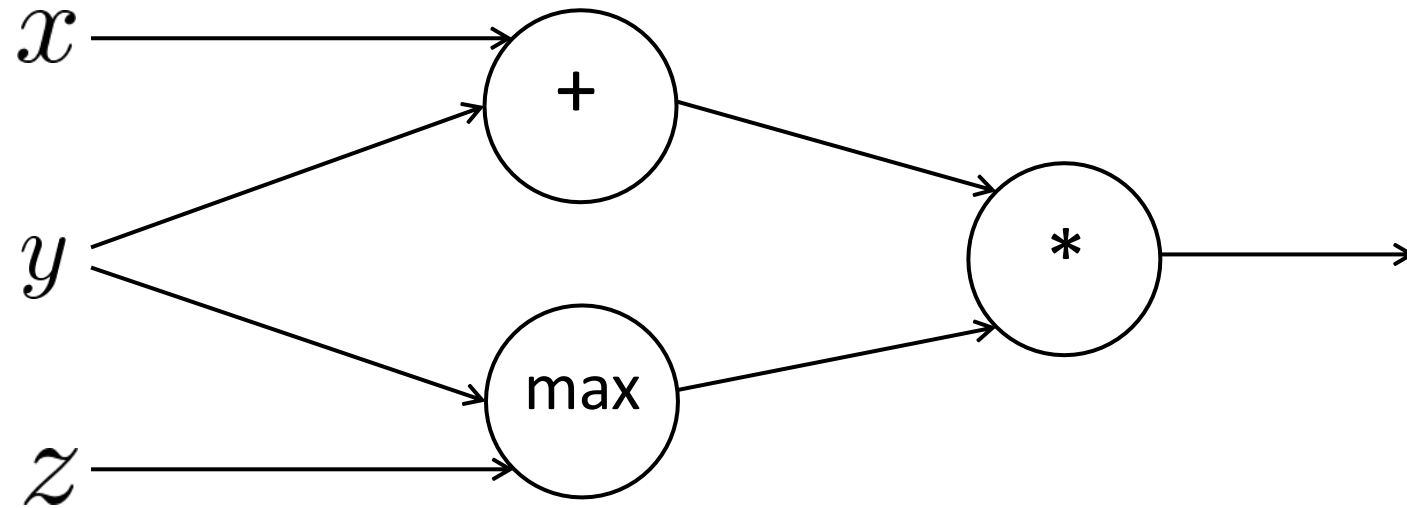
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

$$f = ab$$



An Example

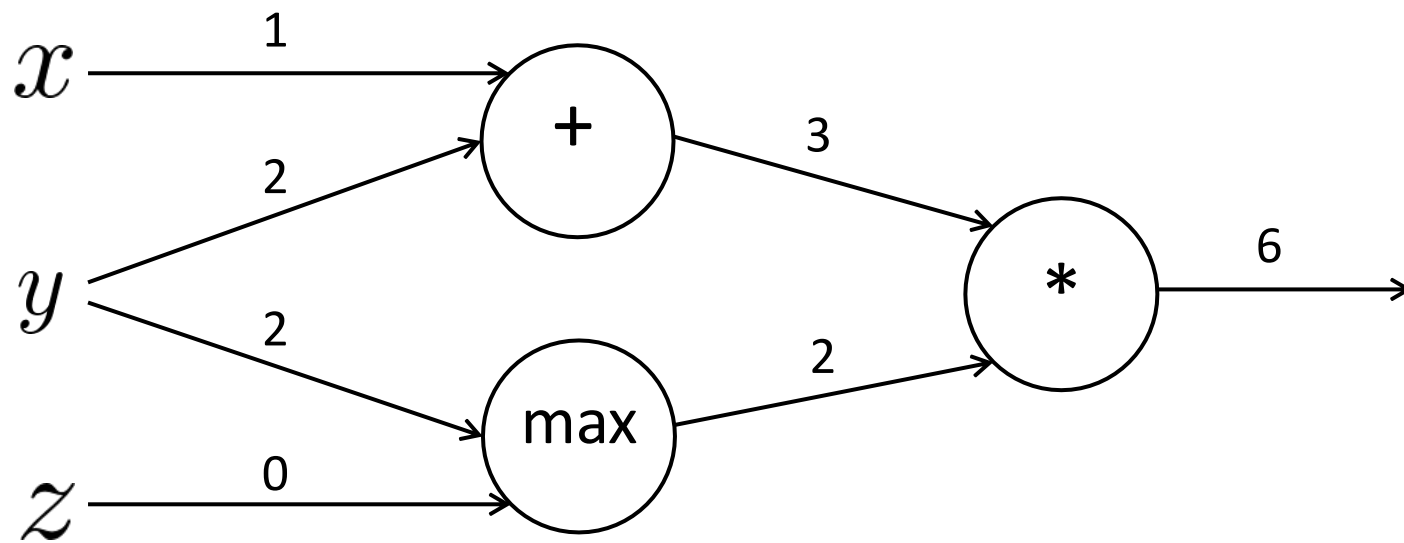
$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

$$f = ab$$



An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

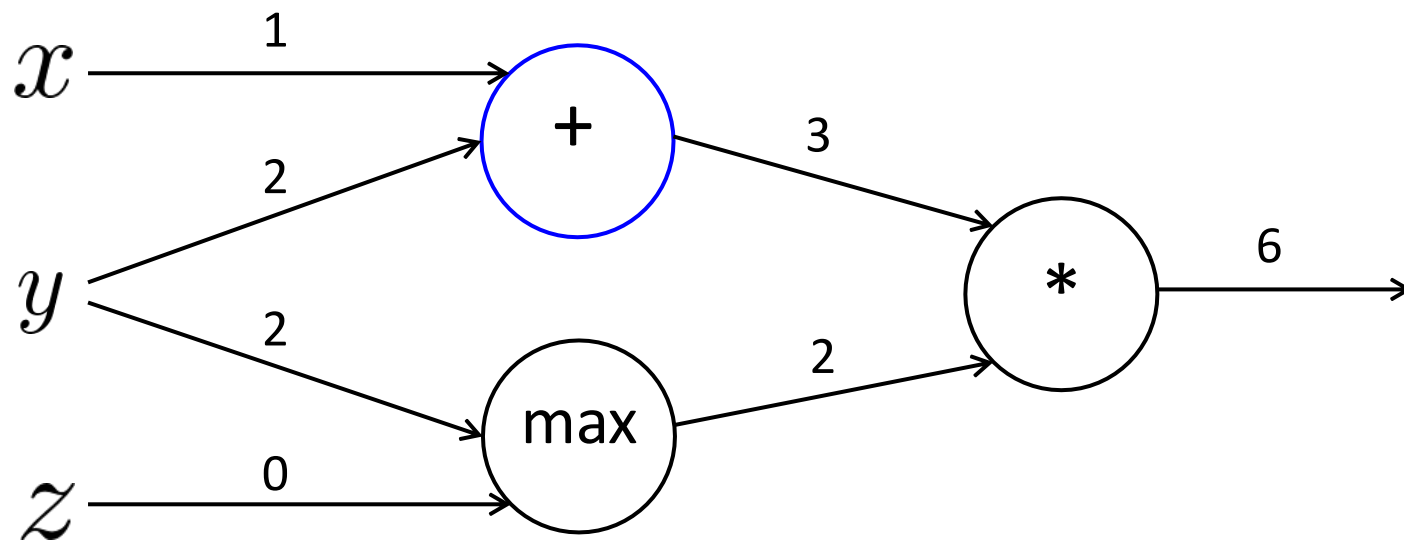
$$a = x + y$$

$$b = \max(y, z)$$

$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$



An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

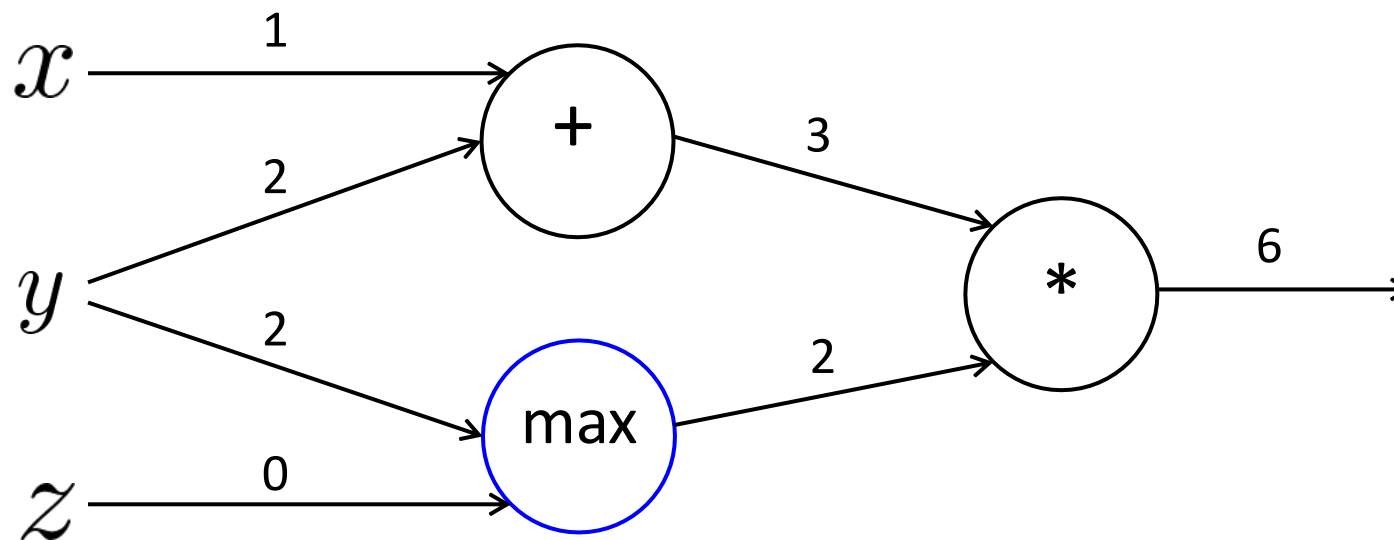
$$b = \max(y, z)$$

$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$



An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

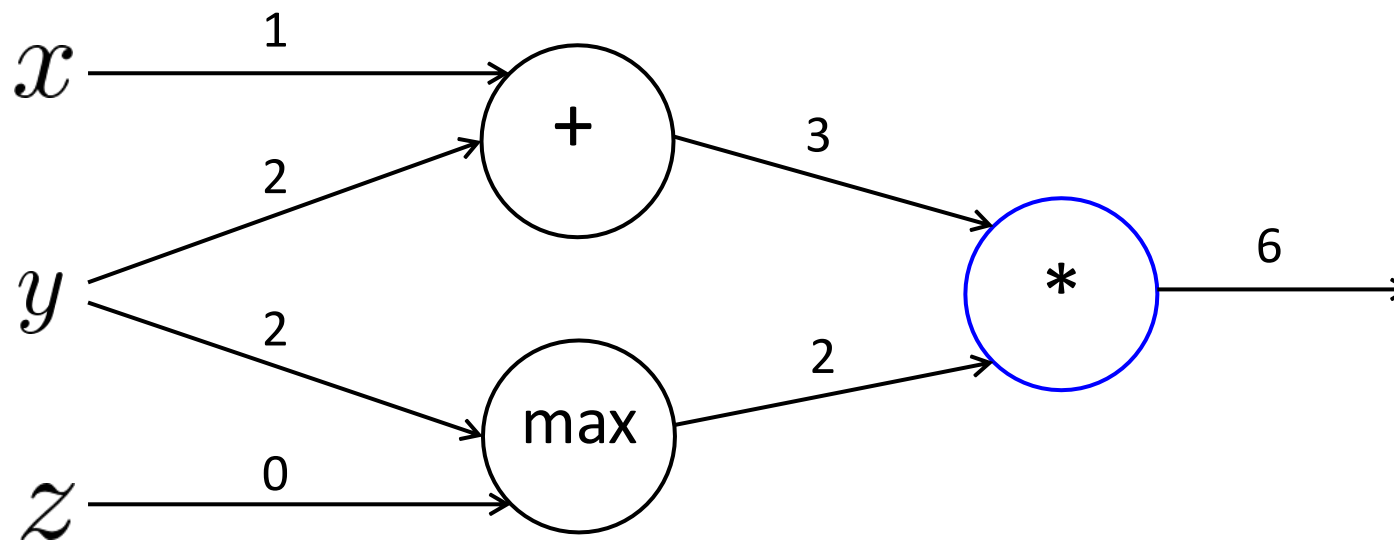
$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$

$$\frac{\partial f}{\partial a} = b = 2 \quad \frac{\partial f}{\partial b} = a = 3$$



An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

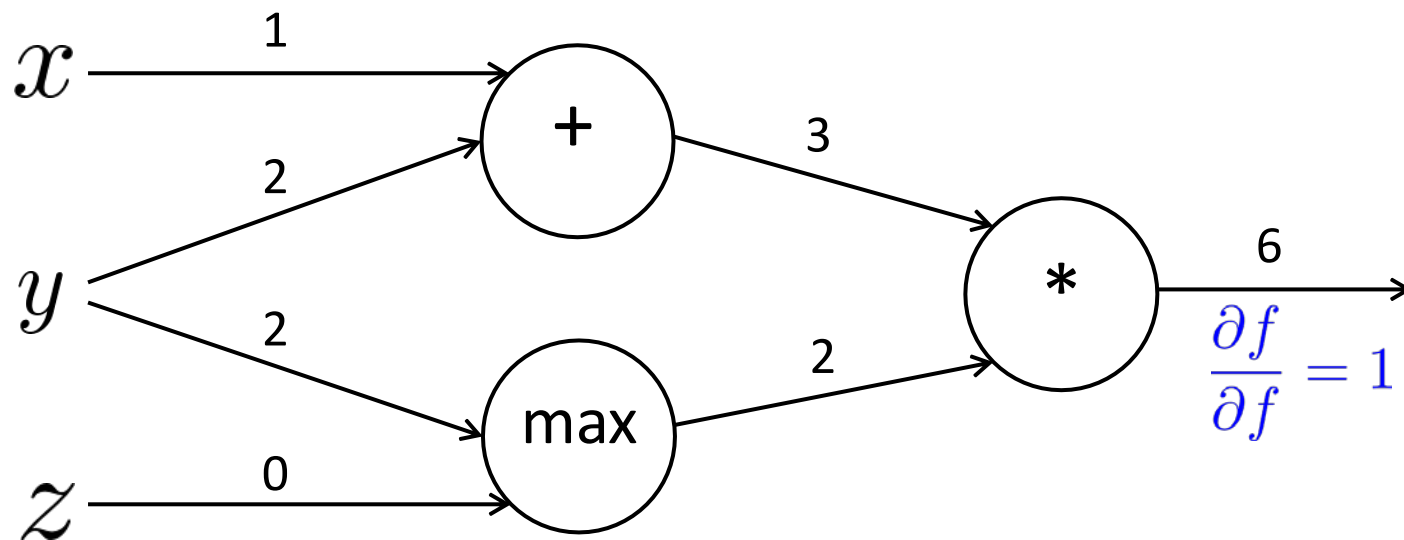
$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$

$$\frac{\partial f}{\partial a} = b = 2 \quad \frac{\partial f}{\partial b} = a = 3$$



An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

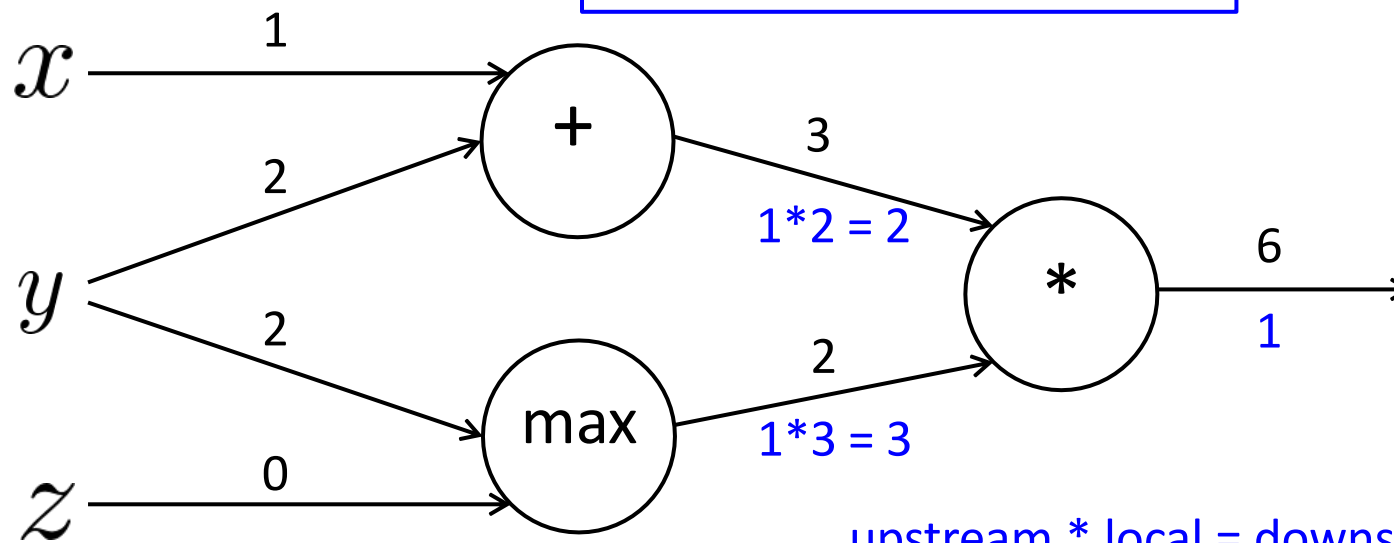
$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$

$$\frac{\partial f}{\partial a} = b = 2 \quad \frac{\partial f}{\partial b} = a = 3$$



upstream * local = downstream

An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

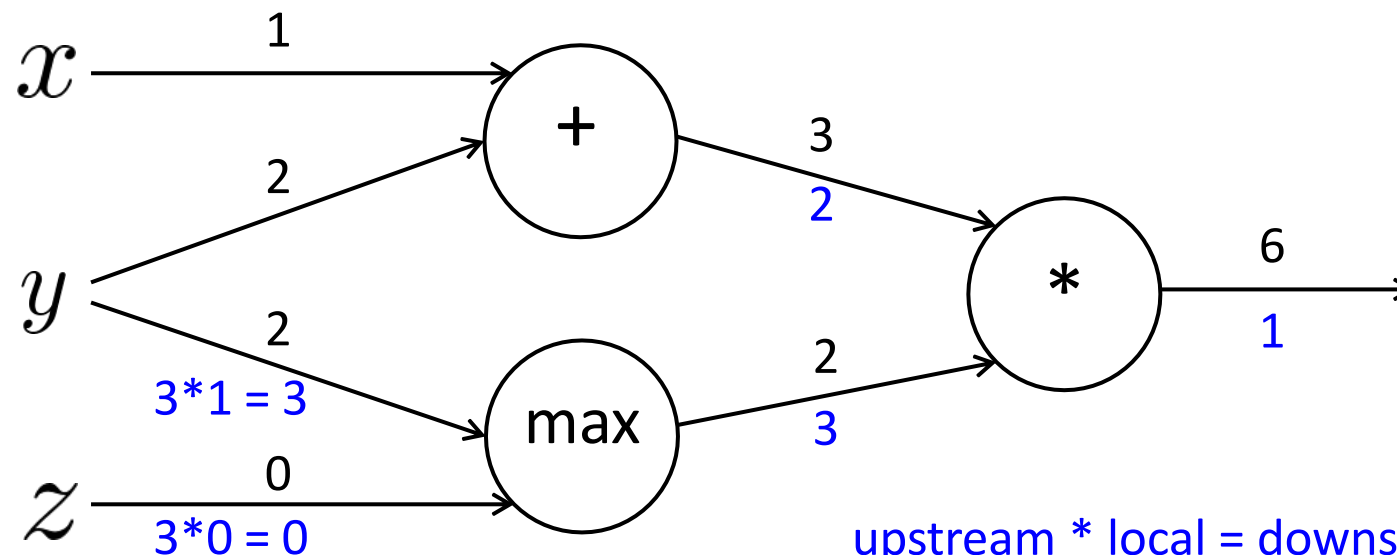
$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$

$$\frac{\partial f}{\partial a} = b = 2 \quad \frac{\partial f}{\partial b} = a = 3$$



upstream * local = downstream

An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

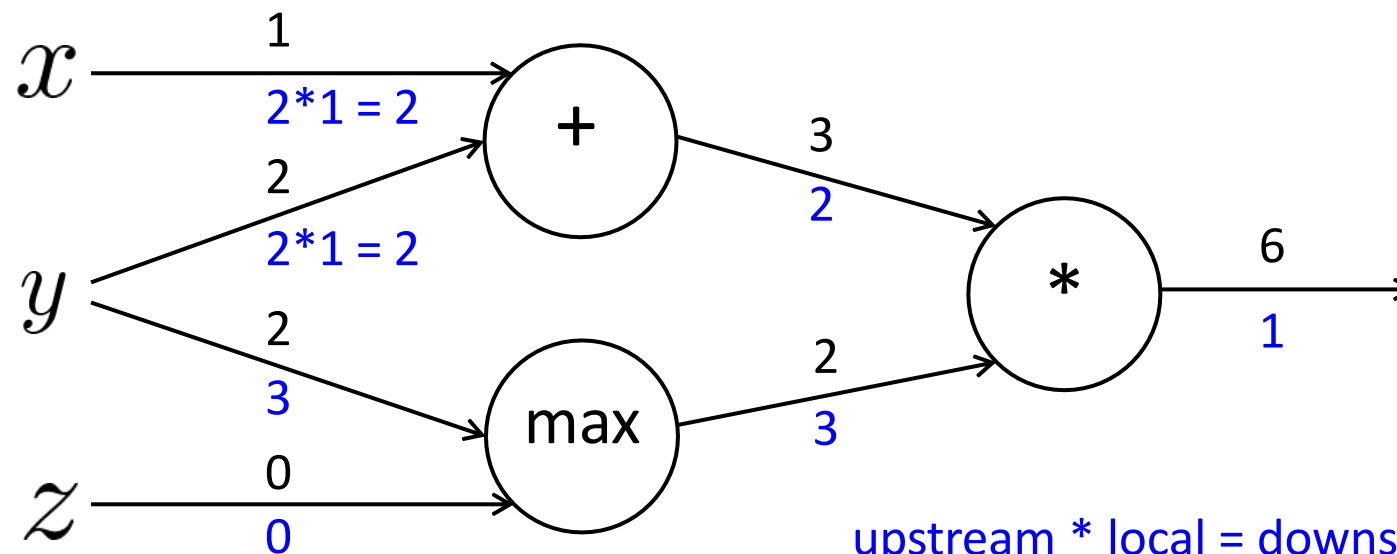
$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$

$$\frac{\partial f}{\partial a} = b = 2 \quad \frac{\partial f}{\partial b} = a = 3$$



An Example

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

Forward prop steps

$$a = x + y$$

$$b = \max(y, z)$$

$$f = ab$$

Local gradients

$$\frac{\partial a}{\partial x} = 1 \quad \frac{\partial a}{\partial y} = 1$$

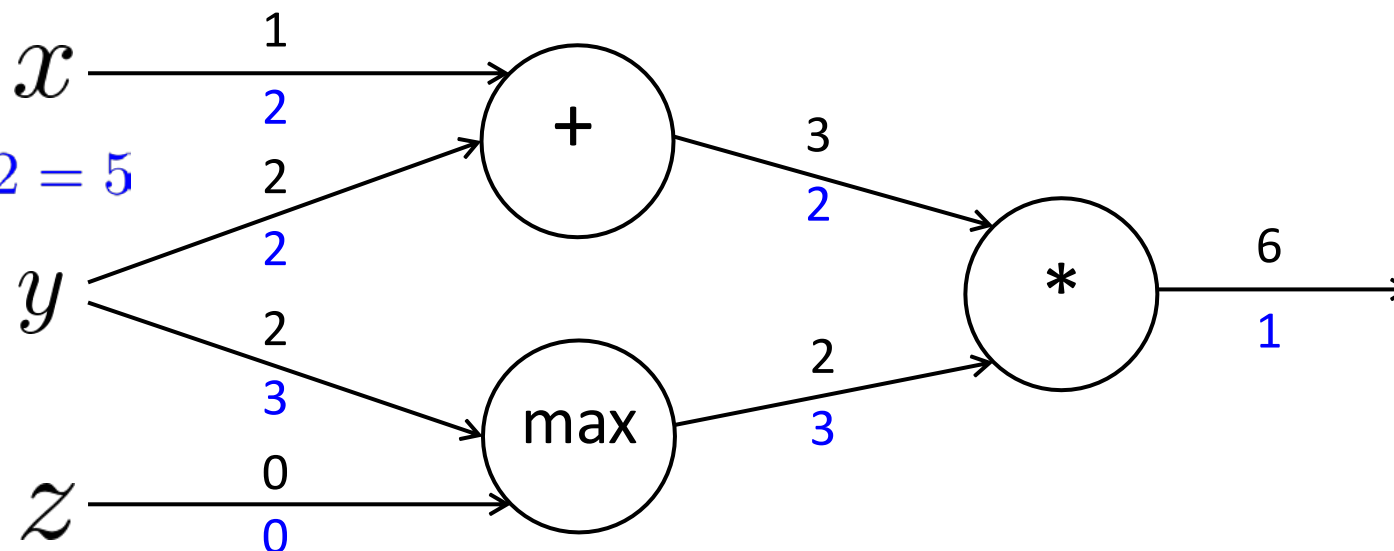
$$\frac{\partial b}{\partial y} = \mathbf{1}(y > z) = 1 \quad \frac{\partial b}{\partial z} = \mathbf{1}(z > y) = 0$$

$$\frac{\partial f}{\partial a} = b = 2 \quad \frac{\partial f}{\partial b} = a = 3$$

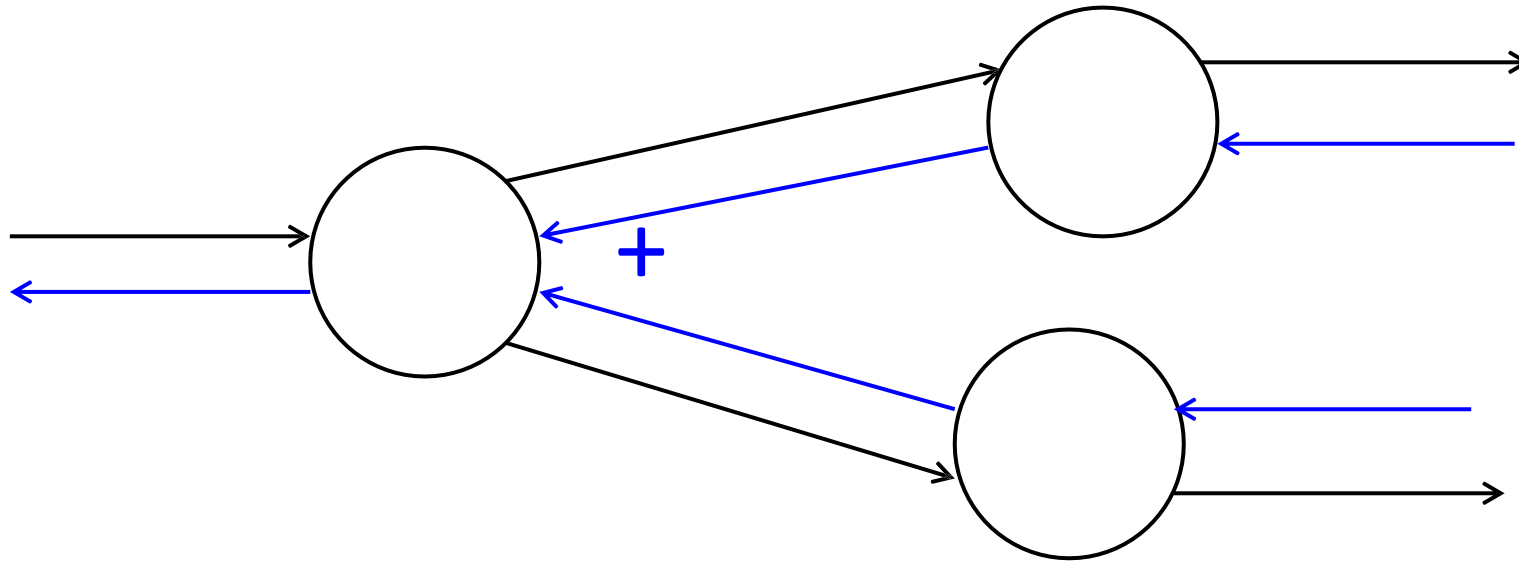
$$\frac{\partial f}{\partial x} = 2$$

$$\frac{\partial f}{\partial y} = 3 + 2 = 5$$

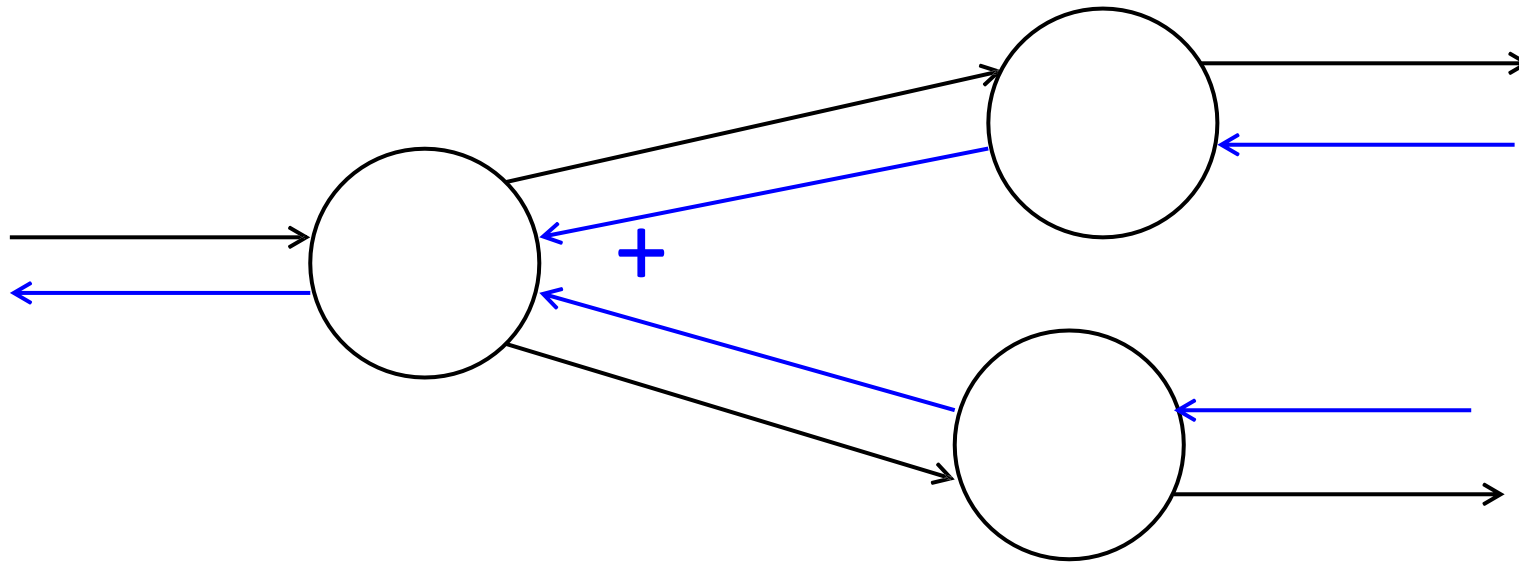
$$\frac{\partial f}{\partial z} = 0$$



Gradients sum at outward branches



Gradients sum at outward branches



$$a = x + y$$

$$b = \max(y, z)$$

$$f = ab$$

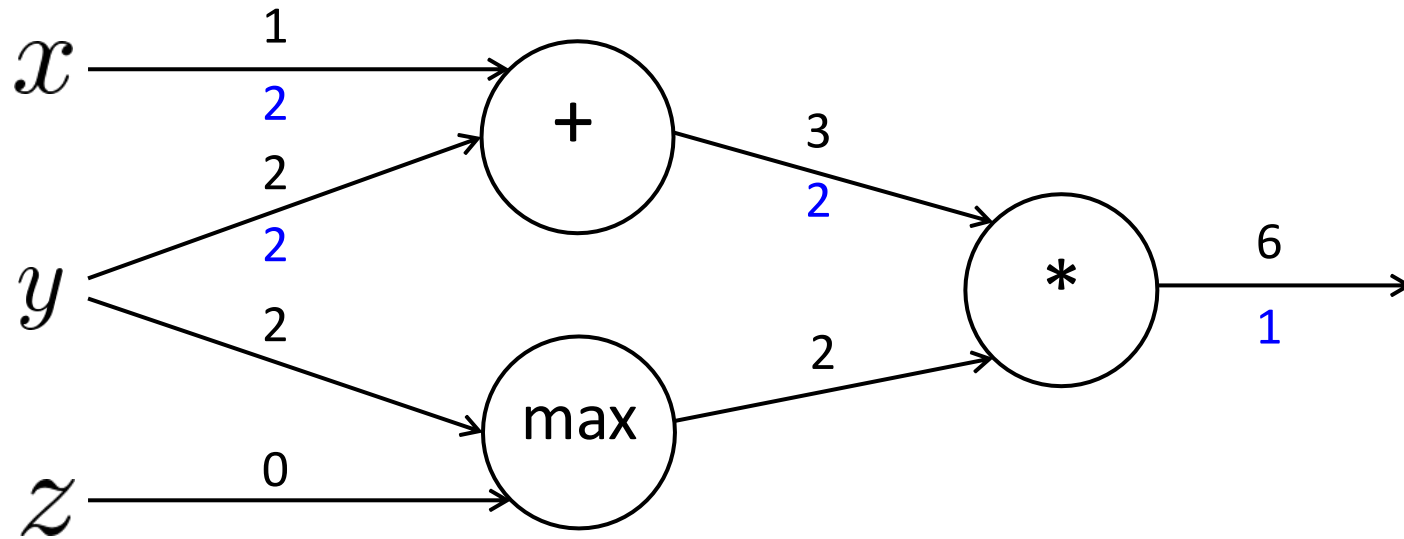
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \frac{\partial a}{\partial y} + \frac{\partial f}{\partial b} \frac{\partial b}{\partial y}$$

Node Intuitions

$$f(x, y, z) = (x + y) \max(y, z)$$

$$x = 1, y = 2, z = 0$$

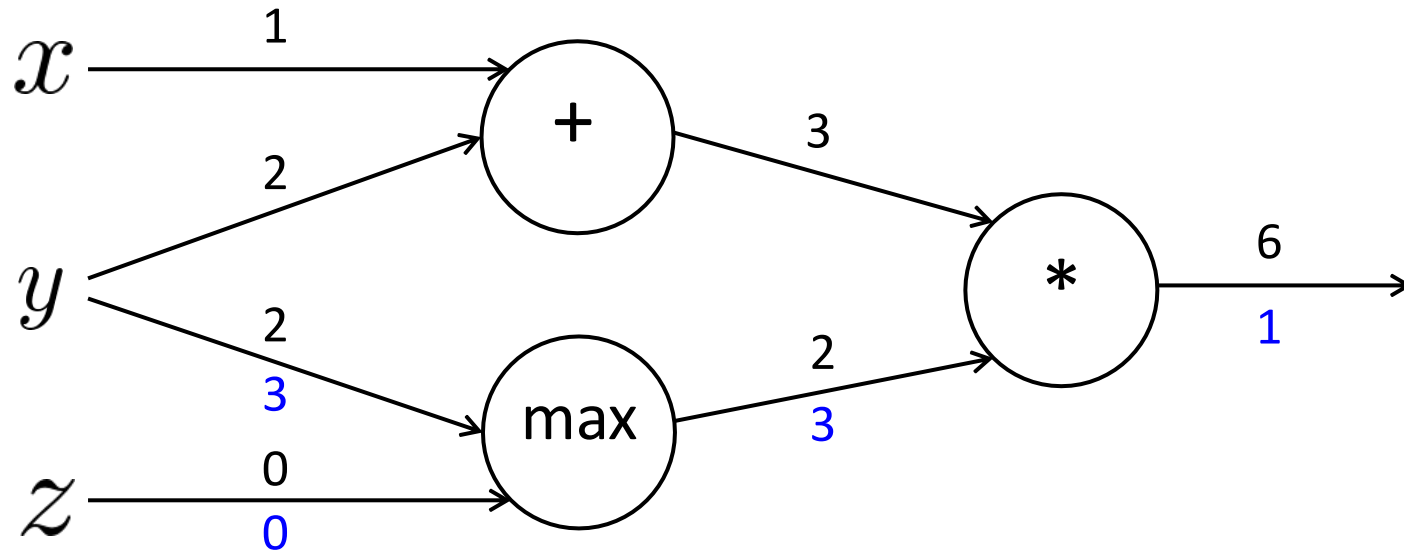
- + “distributes” the upstream gradient to each summand



Node Intuitions

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

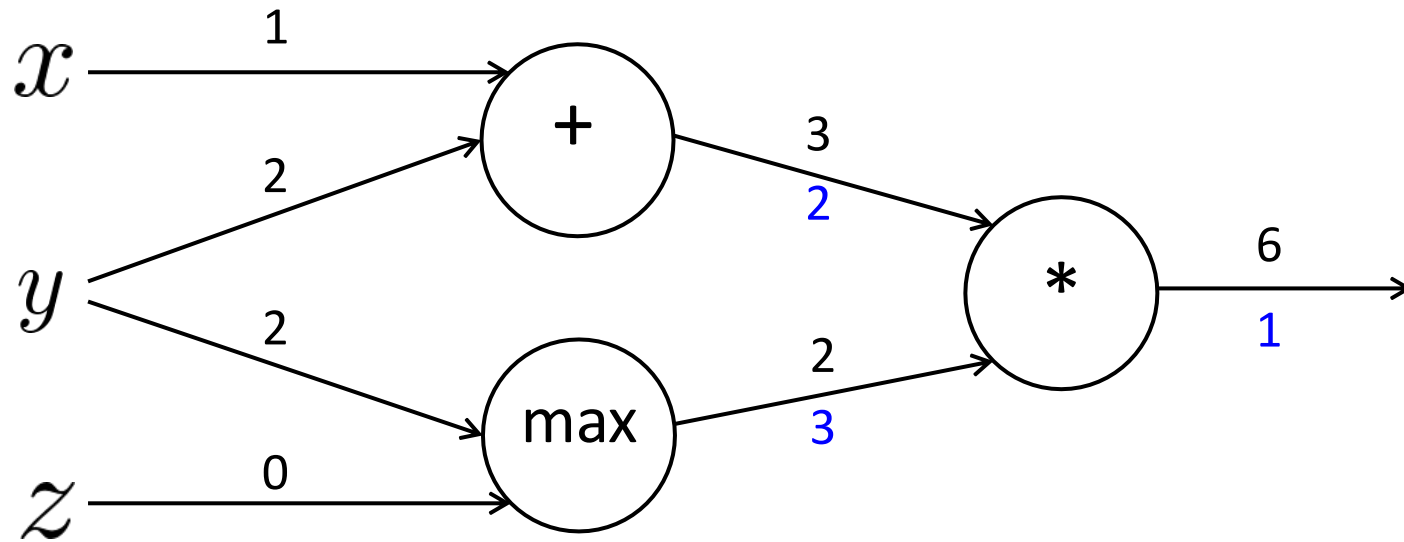
- + “distributes” the upstream gradient to each summand
- max “routes” the upstream gradient



Node Intuitions

$$f(x, y, z) = (x + y) \max(y, z)$$
$$x = 1, y = 2, z = 0$$

- + “distributes” the upstream gradient
- max “routes” the upstream gradient
- * “switches” the upstream gradient



Efficiency: compute all gradients at once

- Incorrect way of doing backprop:

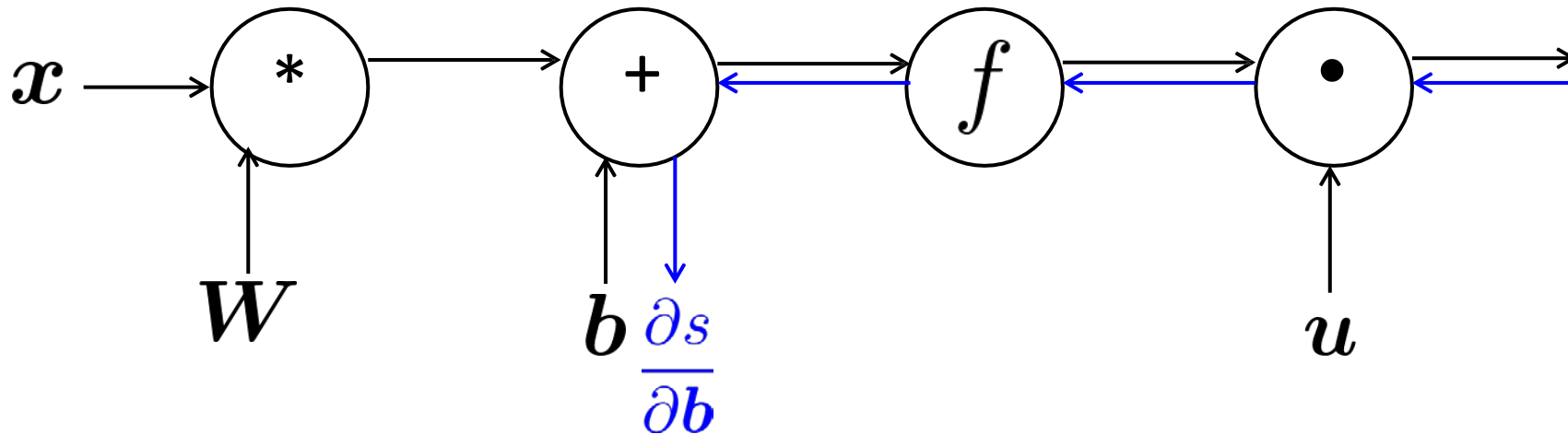
- First compute $\frac{\partial s}{\partial b}$

$$s = u^T h$$

$$h = f(z)$$

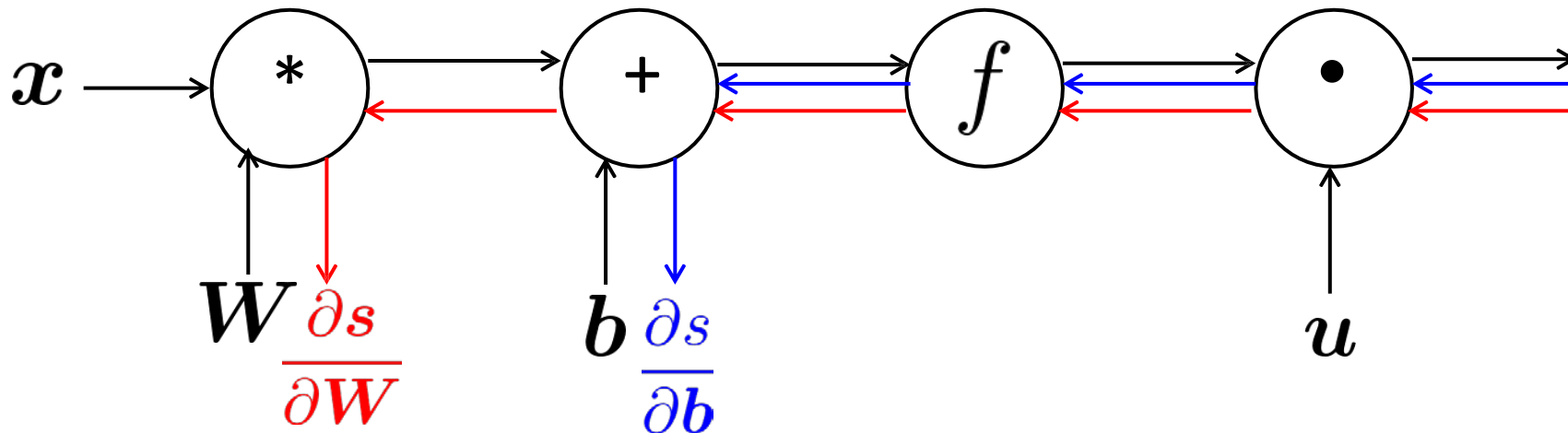
$$z = Wx + b$$

$$x \text{ (input)}$$



Efficiency: compute all gradients at once

- Incorrect way of doing backprop:
 - First compute $\frac{\partial s}{\partial b}$
 - Then independently compute $\frac{\partial s}{\partial W}$
 - Duplicated computation!
- $s = u^T h$
 $h = f(z)$
 $z = Wx + b$
 x (input)



Efficiency: compute all gradients at once

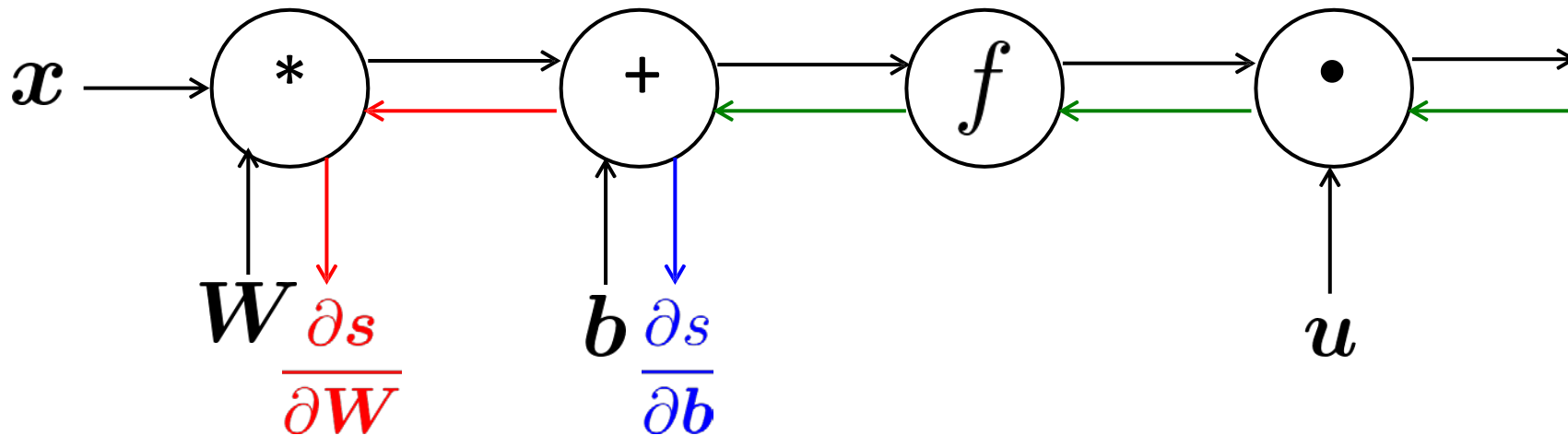
- Correct way:
 - Compute all the gradients at once
 - Analogous to using δ when we computed gradients by hand

$$s = u^T h$$

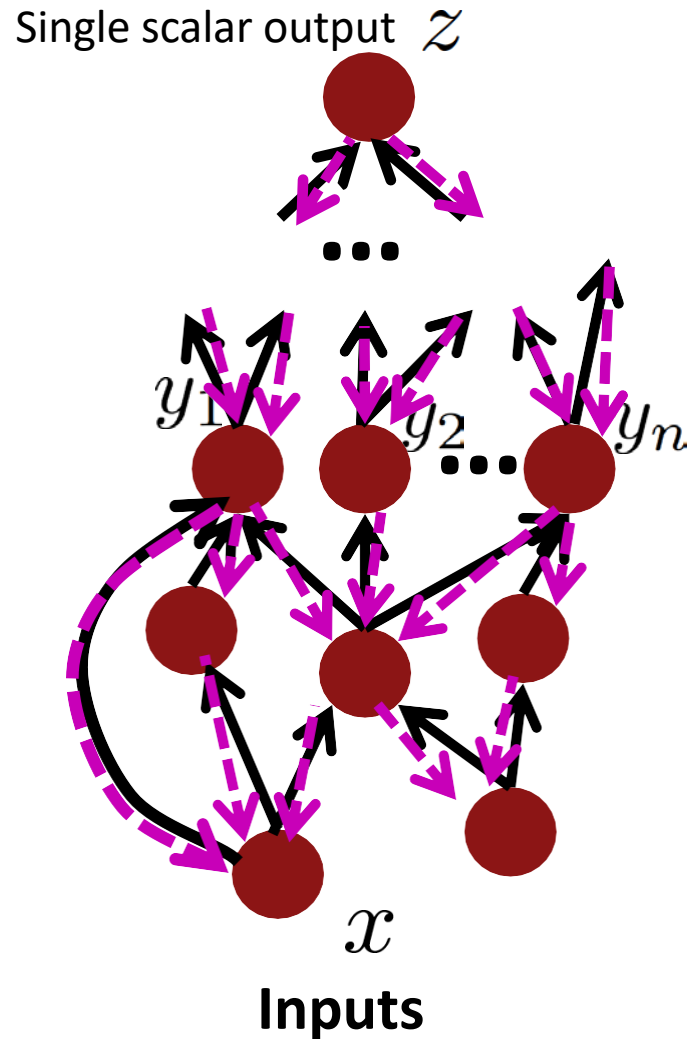
$$h = f(z)$$

$$z = \mathbf{W}x + b$$

$$x \quad (\text{input})$$



Back-Prop in General Computation Graph



1. Fprop: visit nodes in topological sort order
 - Compute value of node given predecessors
2. Bprop:

- initialize output gradient = 1

- visit nodes in reverse order:

Compute gradient wrt each node using
gradient wrt successors

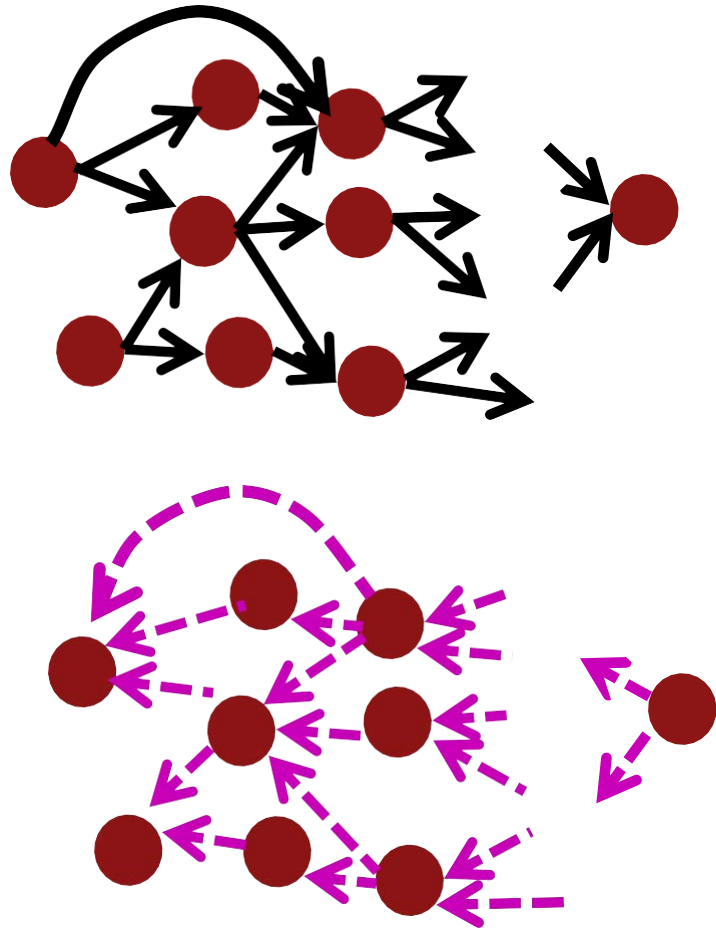
$\{y_1, y_2, \dots, y_n\} = \text{successors of } x$

$$\frac{\partial z}{\partial x} = \sum_{i=1}^n \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x}$$

Done correctly, big $O()$ complexity of fprop and bprop is **the same**

In general, our nets have regular layer-structure and so we can use matrices and Jacobians...

Automatic Differentiation

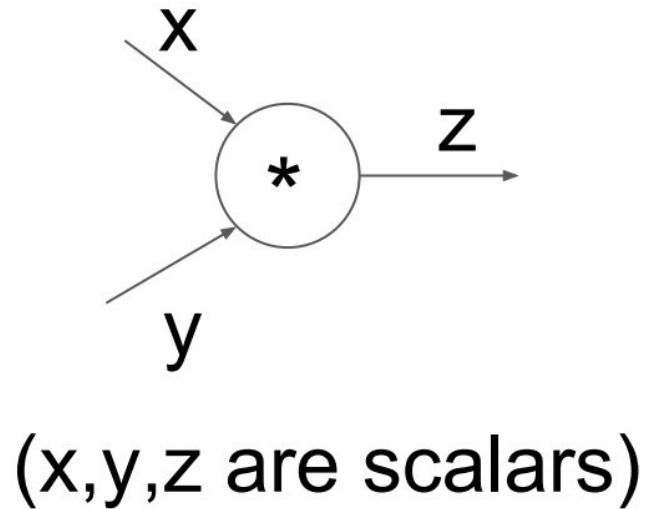


- The gradient computation can be automatically inferred from the symbolic expression of the fprop
- Each node type needs to know how to compute its output and how to compute the gradient wrt its inputs given the gradient wrt its output
- Modern DL frameworks (Tensorflow, PyTorch, etc.) do backpropagation for you but mainly leave layer/node writer to hand-calculate the local derivative

Backprop Implementations

```
class ComputationalGraph(object):  
    #...  
    def forward(inputs):  
        # 1. [pass inputs to input gates...]  
        # 2. forward the computational graph:  
        for gate in self.graph.nodes_topologically_sorted():  
            gate.forward()  
        return loss # the final gate in the graph outputs the loss  
    def backward():  
        for gate in reversed(self.graph.nodes_topologically_sorted()):  
            gate.backward() # little piece of backprop (chain rule applied)  
        return inputs_gradients
```

Implementation: forward/backward API



```
class MultiplyGate(object):  
    def forward(x,y):  
        z = x*y  
        return z  
    def backward(dz):  
        # dx = ... #todo  
        # dy = ... #todo  
        return [dx, dy]
```

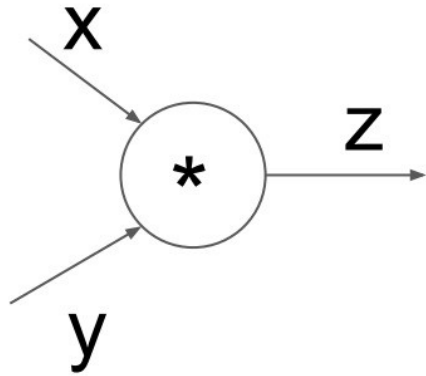
$$\frac{\partial L}{\partial z}$$

An arrow points from this box to the 'dz' parameter in the 'backward' method of the code block.

$$\frac{\partial L}{\partial x}$$

An arrow points from this box to the 'dx' element in the return list of the 'backward' method in the code block.

Implementation: forward/backward API



(x,y,z are scalars)

```
class MultiplyGate(object):  
    def forward(x,y):  
        z = x*y  
        self.x = x # must keep these around!  
        self.y = y  
        return z  
    def backward(dz):  
        dx = self.y * dz # [dz/dx * dL/dz]  
        dy = self.x * dz # [dz/dy * dL/dz]  
        return [dx, dy]
```

Manual Gradient checking: Numeric Gradient

- For small h ($\approx 1\text{e-}4$),
$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}$$
- Easy to implement correctly
- But approximate and **very** slow:
 - You have to recompute f for **every parameter** of our model
- Useful for checking your implementation
 - In the old days, we hand-wrote everything, doing this everywhere was the key test
 - Now much less needed; you can use it to check layers are correctly implemented

Summary

We've mastered the core technology of neural nets! 🎉🎉🎉🎉

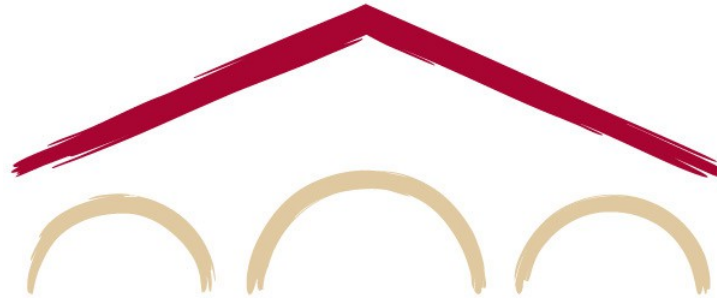
- **Backpropagation:** recursively (and hence efficiently) apply the chain rule along computation graph
 - $[\text{downstream gradient}] = [\text{upstream gradient}] \times [\text{local gradient}]$
- **Forward pass:** compute results of operations and save intermediate values
- **Backward pass:** apply chain rule to compute gradients

Why learn all these details about gradients?

- **Modern deep learning frameworks compute gradients for you!**
- But why take a class on compilers or systems when they are implemented for you?
 - Understanding what is going on under the hood is useful!
- Backpropagation doesn't always work perfectly out of the box
 - Understanding why is crucial for debugging and improving models
 - See Karpathy article (in syllabus):
 - <https://medium.com/@karpathy/yes-you-should-understand-backprop-e2f06eab496b>
 - Example in future lecture: exploding and vanishing gradients

Natural Language Processing with Deep Learning

CS224N/Ling284



Christopher Manning
Lecture 4: Dependency Parsing

Lecture Plan

Syntactic Structure and Dependency parsing

1. Syntactic Structure: Consistency and Dependency (30 mins)
2. Dependency Grammar and Treebanks (15 mins)
3. Transition-based dependency parsing (15 mins)
4. Neural dependency parsing (20 mins)

Key Learnings: Explicit linguistic structure and how a neural net can decide it

1. The linguistic structure of sentences – two views: Constituency = phrase structure grammar = context-free grammars (CFGs)

Phrase structure organizes words into nested constituents

Starting unit: words

the, cat, cuddly, by, door

Words combine into phrases

the cuddly cat, by the door

Phrases can combine into bigger phrases

the cuddly cat by the door

The linguistic structure of sentences – two views: Constituency = phrase structure grammar = context-free grammars (CFGs)

Phrase structure organizes words into nested constituents.

the cat
a dog
large in a crate
barking on the table
cuddly by the door
large barking

talk to

walked behind

Two views of linguistic structure: Dependency structure

- Dependency structure shows which words depend on (modify, attach to, or are arguments of) which other words.

Look in the large crate in the kitchen by the door

Why is sentence structure needed for communication?

Humans communicate complex ideas by composing words together into bigger units to convey complex meanings

Human listeners need to work out what modifies [attaches to] what

A model needs to understand sentence structure in order to be able to interpret language correctly

Prepositional phrase attachment ambiguity

San Jose cops kill man with knife

Text Paper Translate Listen Close

BBC Sign in News Sport Weather Shop Reel Travel

NEWS

Home Video World US & Canada UK Business Tech Science Stories

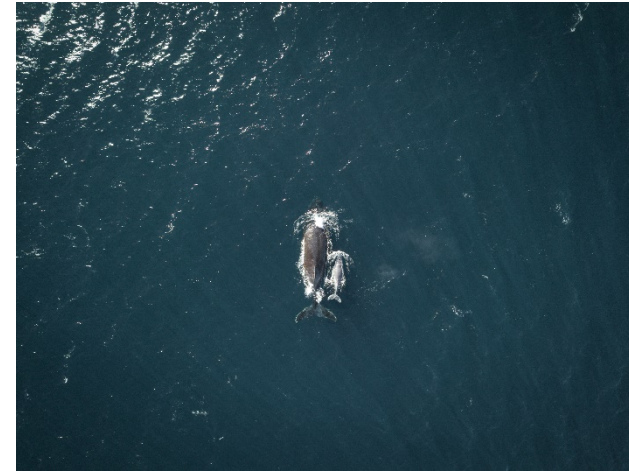
Science & Environment

Scientists count whales from space

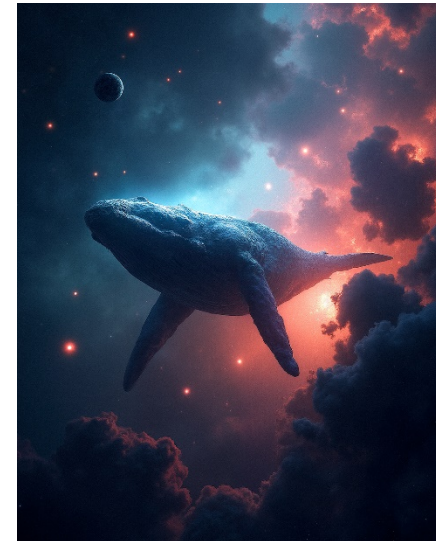
By Jonathan Amos
BBC Science Correspondent

Prepositional phrase attachment ambiguity

Scientists count whales from space



Scientists count whales from space



PP attachment ambiguities multiply

- A key parsing decision is how we ‘attach’ various constituents
 - PPs, adverbial or participial phrases, infinitives, coordinations,

The board approved [its acquisition] [by Royal Trustco Ltd.]
[of Toronto]
[for \$27 a share]
[at its monthly meeting].

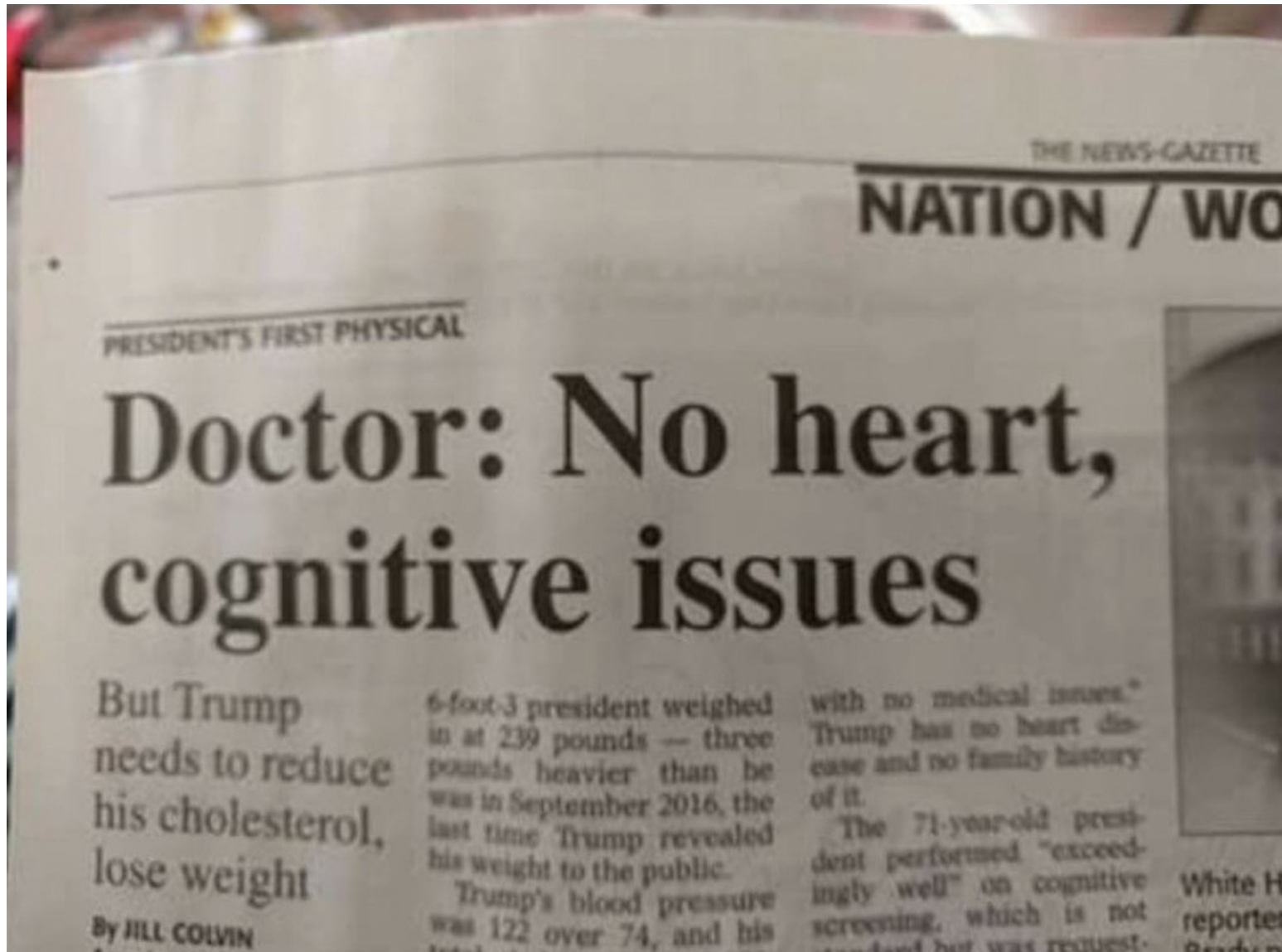
- Catalan numbers: $C_n = (2n)! / [(n+1)!n!]$
- An exponentially growing series, which arises in many tree-like contexts:
 - E.g., the number of possible triangulations of a polygon with $n+2$ sides
 - Turns up in triangulation of probabilistic graphical models (CS228)....

Coordination scope ambiguity

Shuttle veteran and longtime NASA executive Fred Gregory appointed to board

Shuttle veteran and longtime NASA executive Fred Gregory appointed to board

Coordination scope ambiguity



Adjectival/Adverbial Modifier Ambiguity

numbers, including some that featured a bucket and bells brigade of performers who beat rhythms on buckets and trash cans with drums sticks and hammer mallets. PHOTO BY JENNIFER STULTZ

MENTORING DAY Students get first hand job experience

By Gale Rose
grose@pratttribune.com

Eager students invaded businesses all over Pratt Tuesday, October 24 as they looked for future job opportunities on Disability Mentoring Day.

The 97 students from 12 schools fanned out across Pratt and got first hand experience what it would be like to work at those 40 businesses. They asked questions and got some hands on experience with various operations.

Paola Luna of Pratt High School, Gina Patton of Kingman High School and America Fernandez of St. John chose the Main Street Small Animal Veterinarian Clinic for their business. Students got a tour of the facility, learned what happens in an examination, got to handle various animals and watched a snake eat a mouse.

Luna said she was interested in animal health and wanted to know more about caring for hurt animals. Patton likes all kinds of animals and said she learned a lot from the experience. Watching the snake eat the mouse impressed her the most.

Fernandez wants to become a veterinarian and enjoyed learning everything that veterinarians

SEE MENTORING, 6

ing Meyer ty Commissioner

- Hospital Pharmacist for 41 years
- 4 years Commissioner for Pratt Planning and Zoning Board of Appeals
- 3 years Pratt City Commission
- Graduate of Pratt High School and KU School of Pharmacy
- Past Member and President of Civic Groups and Organizations
- Experience and Knowledge of Financial Responsibility and Budgeting
- Supports Family Values, Education, and Business Growth
- Common Sense Approach for the Sustained Progress of Pratt

12 SATURDAY, October 28, 2017 ■ The Pratt Tribune ■ www.pratttribune.com

Verb Phrase (VP) attachment ambiguity



The screenshot shows the top navigation bar of The Guardian website. It includes a dark blue header with a user profile icon, a search icon, and a menu icon (three dots). The word "theguardian" is displayed in white. Below this is a light gray navigation bar with links for "home", "world", "americas", and "asia", followed by a dark gray button with a hamburger menu icon and the text "all". The main content area features the section title "Rio de Janeiro" in bold blue text, followed by a horizontal dotted line. The headline "Mutilated body washes up on Rio beach to be used for Olympics beach volleyball" is displayed in a large, black, serif font.

theguardian

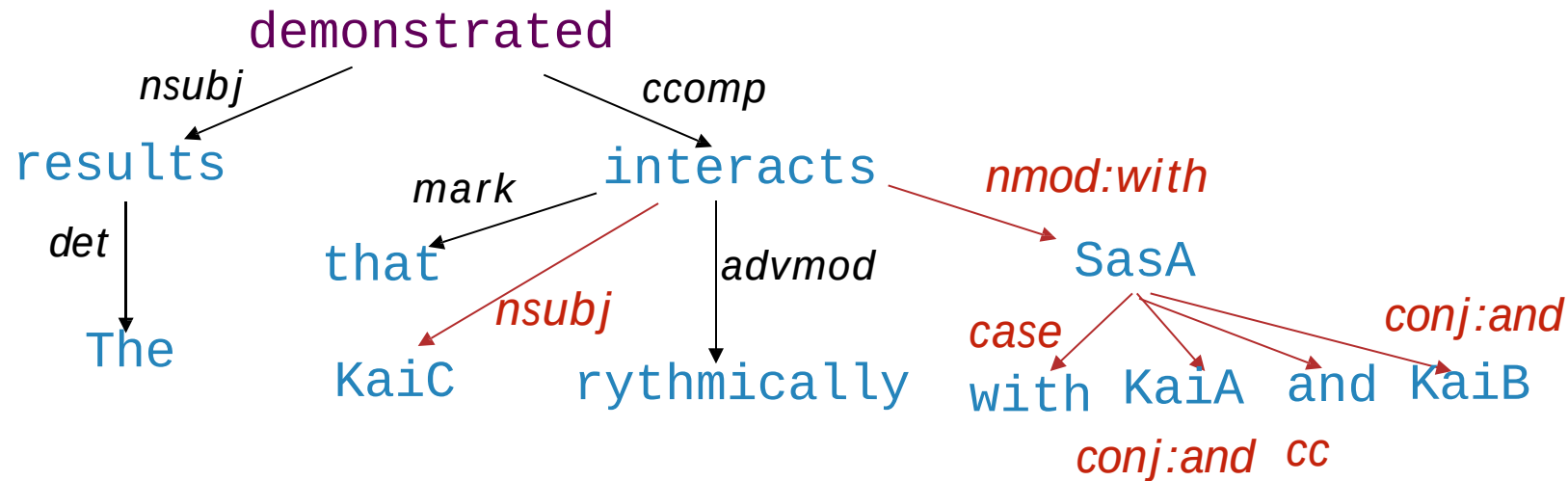
home › world › americas asia ≡ all

Rio de Janeiro

Mutilated body washes up
on Rio beach to be used for
Olympics beach volleyball

6/29/16, 1:48 PM

Dependency paths help extract semantic interpretation – simple practical example: extracting protein-protein interaction



KaiC $\leftarrow$ nsubj interacts nmod:with $\rightarrow$ SasA

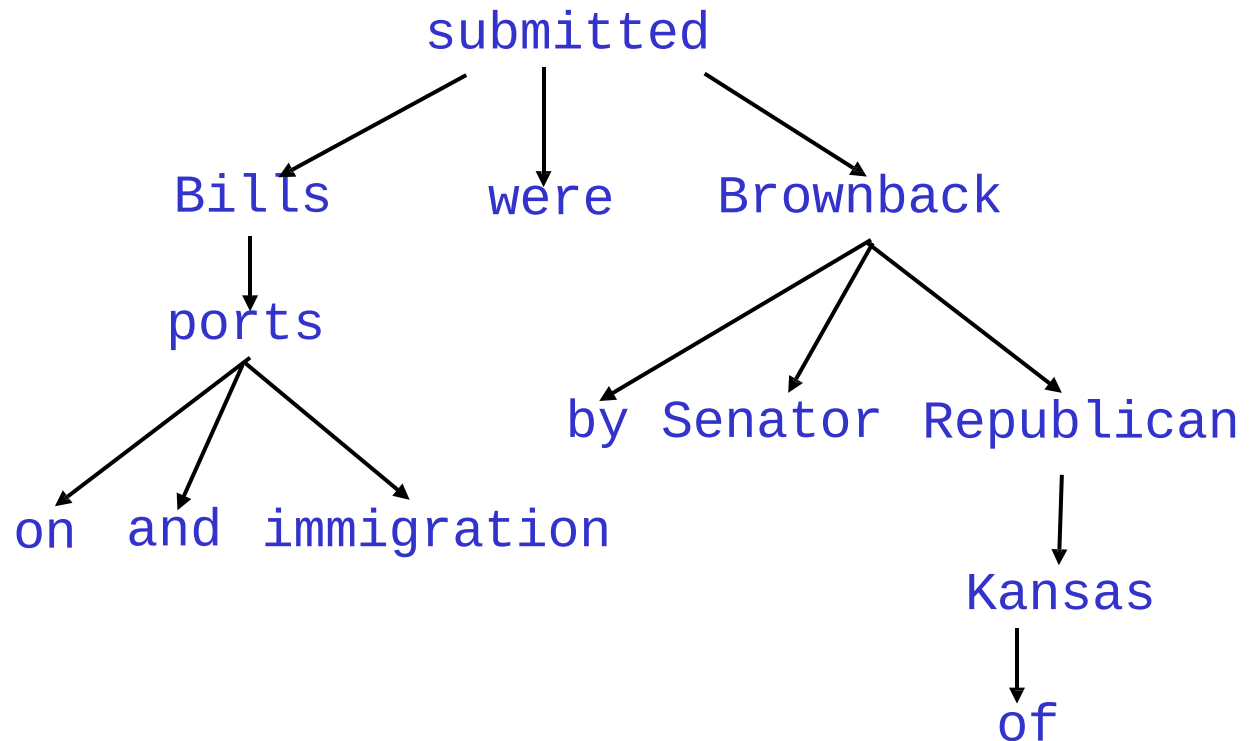
KaiC $\leftarrow$ nsubj interacts nmod:with $\rightarrow$ SasA conj:and $\rightarrow$ KaiA

KaiC $\leftarrow$ nsubj interacts nmod:with $\rightarrow$ SasA conj:and $\rightarrow$ KaiB

[Erkan et al. EMNLP 07, Fundel et al. 2007, etc.]

2. Dependency Grammar and Dependency Structure

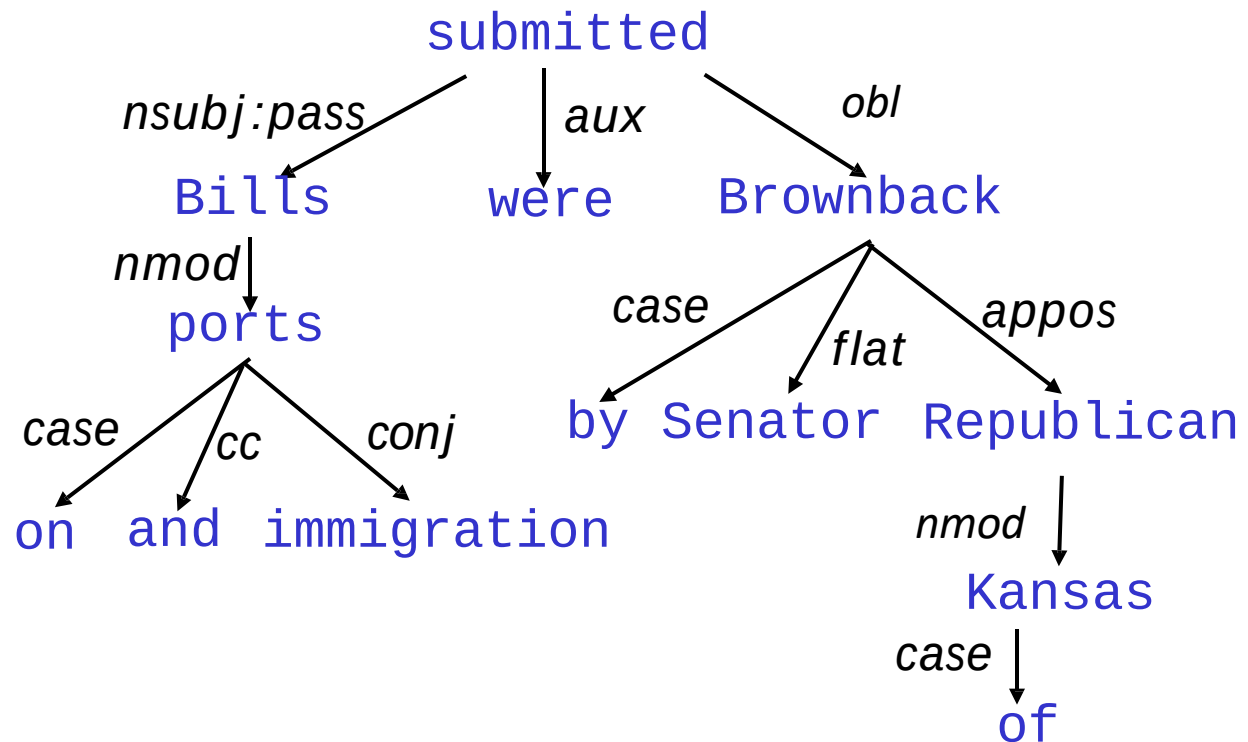
Dependency syntax postulates that syntactic structure consists of relations between lexical items, normally binary asymmetric relations (“arrows”) called **dependencies**



Dependency Grammar and Dependency Structure

Dependency syntax postulates that syntactic structure consists of relations between lexical items, normally binary asymmetric relations (“arrows”) called **dependencies**

The arrows are commonly **typed** with the name of grammatical relations (subject, prepositional object, apposition, etc.)

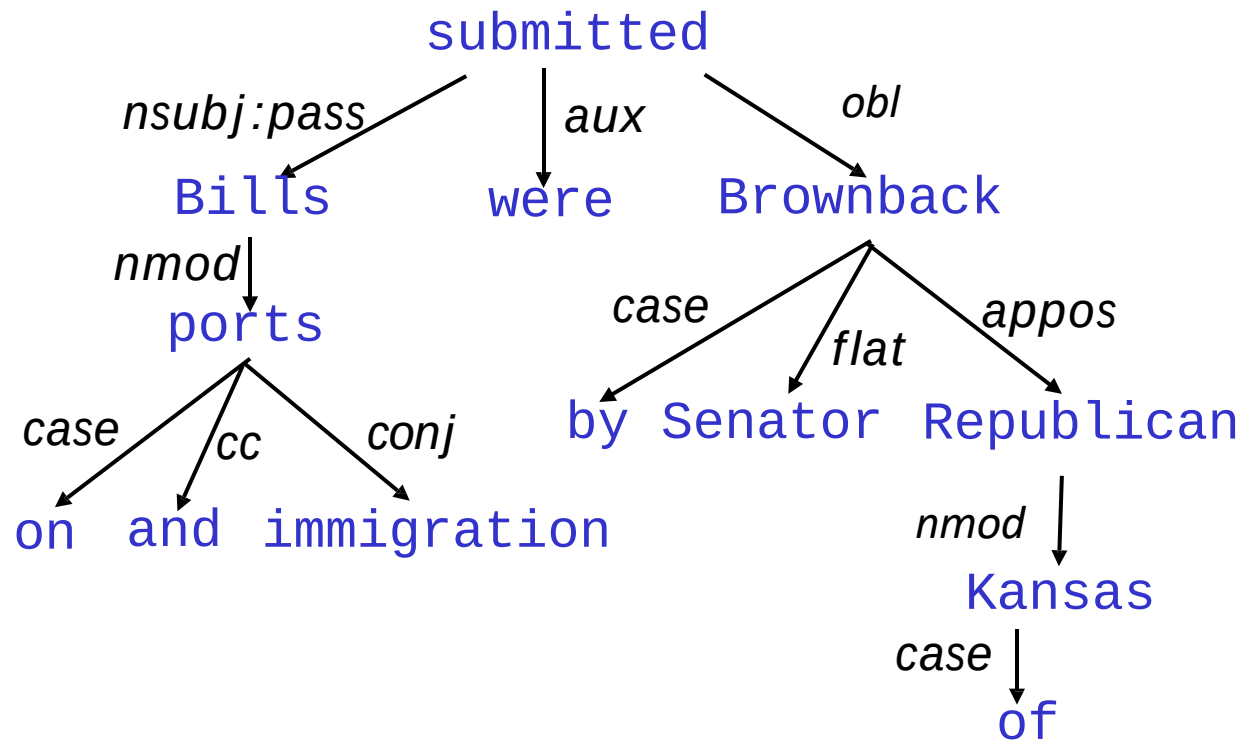


Dependency Grammar and Dependency Structure

Dependency syntax postulates that syntactic structure consists of relations between lexical items, normally binary asymmetric relations (“arrows”) called **dependencies**

An arrow connects a **head** (governor, superior, regent) with a **dependent** (modifier, inferior, subordinate)

Usually, dependencies form a tree (a connected, acyclic, single-root graph)



Dependency Grammar/Parsing History

- The idea of dependency structure goes back a long way
 - To Pāṇini's grammar (composed c. 5th century BCE)
 - Basic approach of 1st millennium Arabic grammarians
- Constituency/context-free grammar is a new-fangled invention
 - 20th century invention (R.S. Wells, 1947; then Chomsky 1953, etc.)
- Modern dependency work is often sourced to Lucien Tesnière (1959)
 - Was dominant approach in “East” in 20th Century (Russia, China, ...)
 - Good for free-er word order, inflected languages like Russian (or Latin!)
- Used in some of the earliest parsers in NLP, even in the US:
 - David Hays, one of the founders of U.S. computational linguistics, built early (first?) dependency parser (Hays 1962) and published on dependency grammar in *Language*

Pāṇini's grammar (composed c. 5th century BCE)

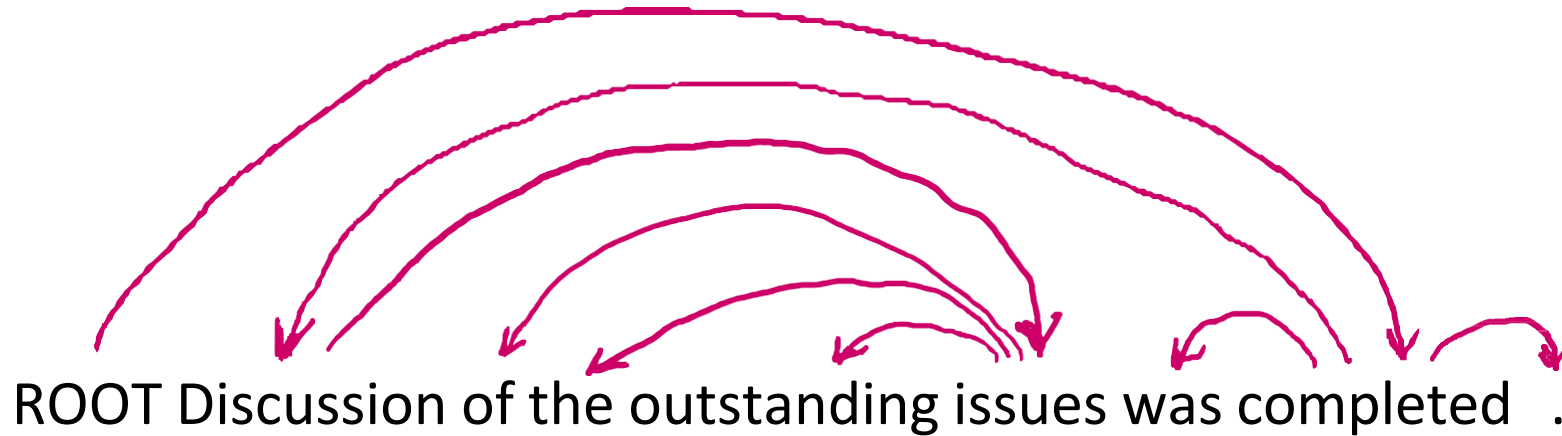


Gallery: <http://wellcomeimages.org/indexplus/image/L0032691.html>

CC BY 4.0 File: Birch bark MS from Kashmir of the Rupavatra Wellcome L0032691.jpg

But this comes from much later – originally the grammar was oral

Dependency Grammar and Dependency Structure

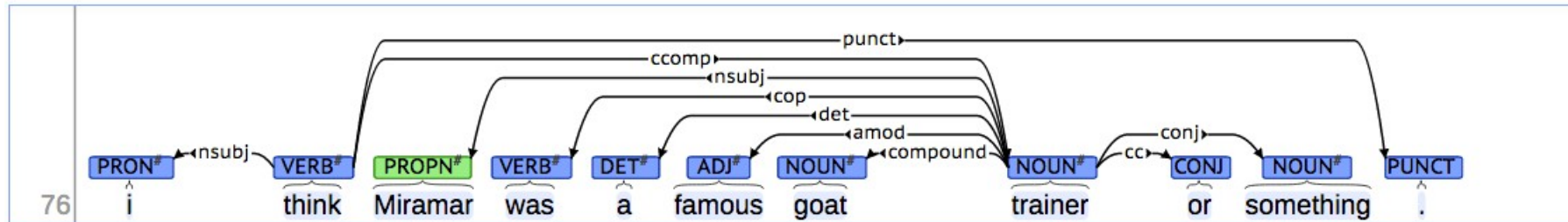


- Some people draw the arrows one way; some the other way!
 - Tesnière had them point from head to dependent – we follow that convention
- We usually add a fake ROOT so every word is a dependent of precisely 1 other node

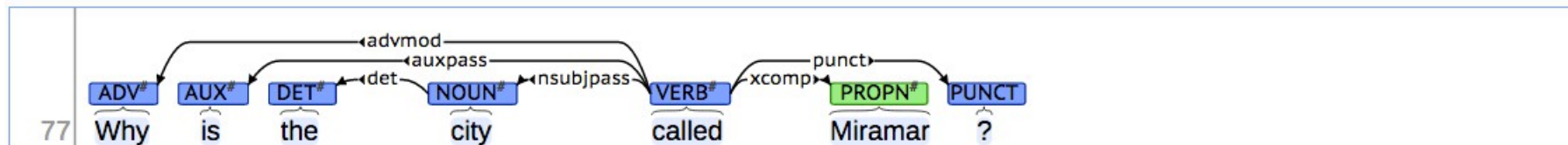
The rise of annotated data & Universal Dependencies treebanks

Brown corpus (1967; PoS tagged 1979); Lancaster-IBM Treebank (starting late 1980s);
Marcus et al. 1993, The Penn Treebank, *Computational Linguistics*;
Universal Dependencies: <http://universaldependencies.org/>

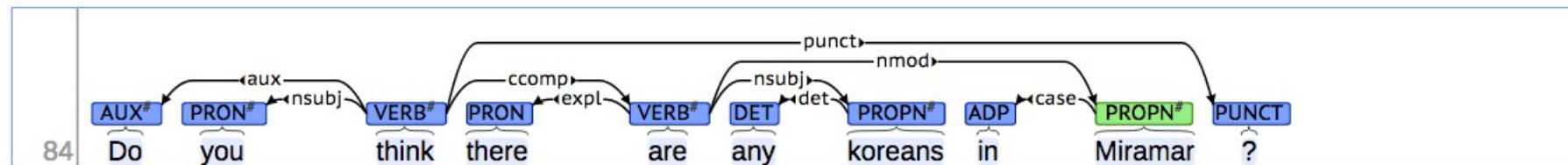
[context] [conllu]



[context] [conllu]



[context] [conllu]



The rise of annotated data

Starting off, building a treebank seems a lot slower and less useful than writing a grammar (by hand)

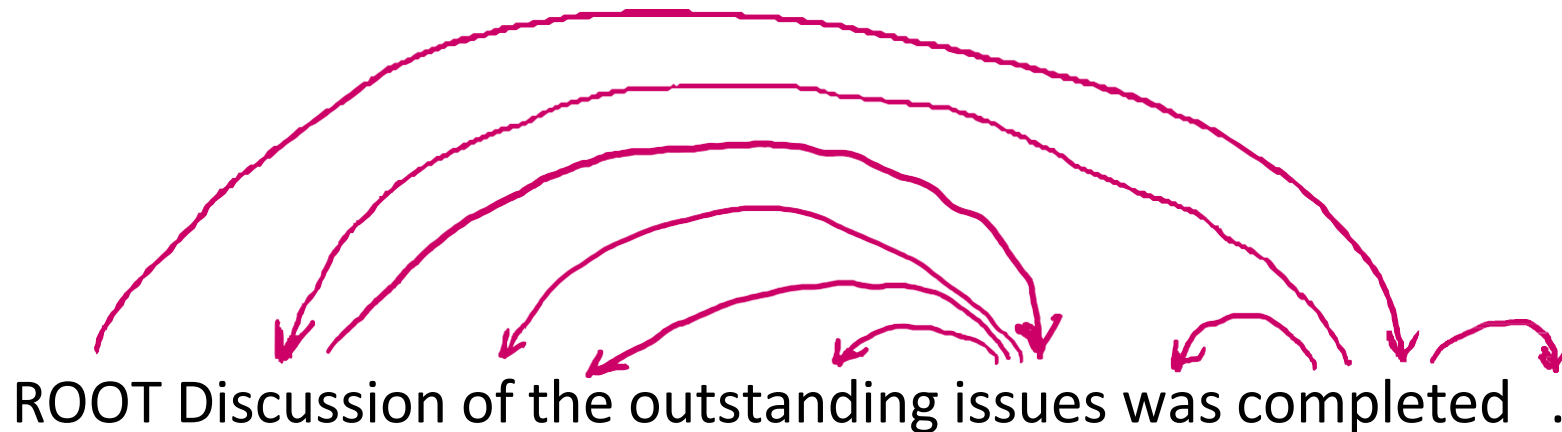
But a treebank gives us many things

- Reusability of the labor
 - Many parsers, part-of-speech taggers, etc. can be built on it
 - Valuable resource for linguistics
- Broad coverage, not just a few intuitions
- Frequencies and distributional information
- A way to evaluate NLP systems

Dependency Conditioning Preferences

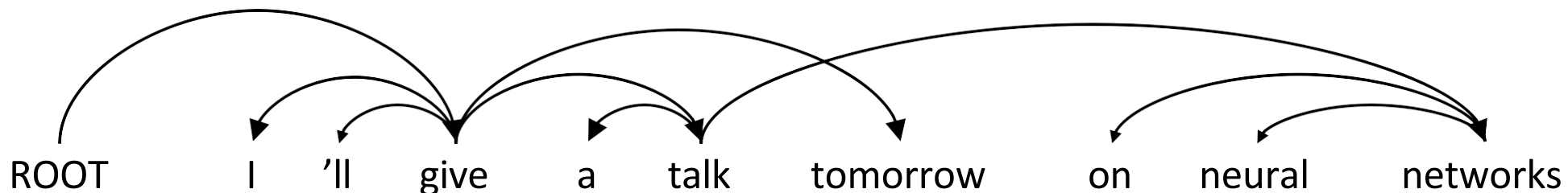
What are the straightforward sources of information for dependency parsing?

1. Bilexical affinities The dependency [discussion → issues] is plausible
2. Dependency distance Most (but not all) dependencies are between nearby words
3. Intervening material Dependencies rarely span intervening verbs or punctuation
4. Valency of heads How many dependents on which side are usual for a head?



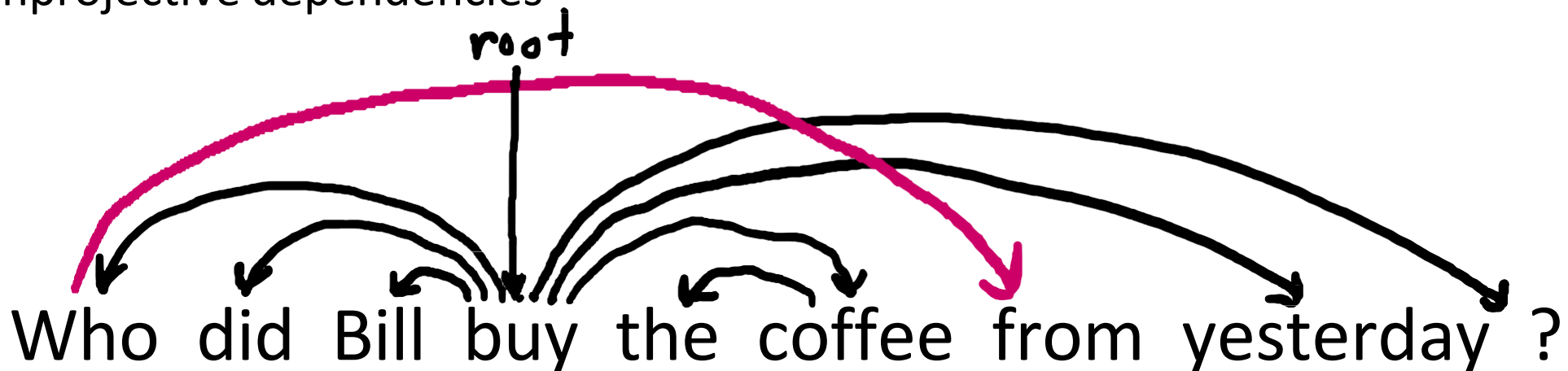
Dependency Parsing

- A sentence is parsed by choosing for each word what other word (including ROOT) it is a dependent of
- Usually some constraints:
 - Only one word is a dependent of ROOT
 - Don't want cycles $A \rightarrow B, B \rightarrow A$
- This makes the dependencies a tree
- Final issue is whether arrows can cross (be **non-projective**) or not



Projectivity

- Definition of a **projective parse**: There are no crossing dependency arcs when the words are laid out in their linear order, with all arcs above the words
- Dependencies corresponding to a CFG tree must be **projective**
 - I.e., by forming dependencies by taking 1 child of each category as head
- Most syntactic structure is projective like this, but dependency theory normally does allow non-projective structures to account for displaced constituents
 - You can't easily get the semantics of certain constructions right without these nonprojective dependencies



3. Methods of Dependency Parsing

1. Dynamic programming

Eisner (1996) gives a clever algorithm with complexity $O(n^3)$, by producing parse items with heads at the ends rather than in the middle

2. Graph algorithms

You create a Minimum Spanning Tree for a sentence

McDonald et al.'s (2005) $O(n^2)$ MSTParser scores dependencies independently using an ML classifier (he uses MIRA, for online learning, but it can be something else)

Neural graph-based parser: Dozat and Manning (2017) et seq. – very successful!

3. Constraint Satisfaction

Edges are eliminated that don't satisfy hard constraints. Karlsson (1990), etc.

4. “Transition-based parsing” or “deterministic dependency parsing”

Greedy choice of attachments guided by good machine learning classifiers

E.g., MaltParser (Nivre et al. 2008). Has proven highly effective. And fast.

Greedy transition-based parsing [Nivre 2003]

- A simple form of a greedy discriminative dependency parser
- The parser does a sequence of bottom-up actions
 - Roughly like “shift” or “reduce” in a shift-reduce parser – CS143, anyone?? – but the “reduce” actions are specialized to create dependencies with head on left or right
- The parser has:
 - a stack σ , written with top to the right
 - which starts with the ROOT symbol
 - a buffer β , written with top to the left
 - which starts with the input sentence
 - a set of dependency arcs A
 - which starts off empty
 - a set of actions

Arc-standard transition-based parser

(there are other transition schemes ...)

Analysis of “I ate fish”

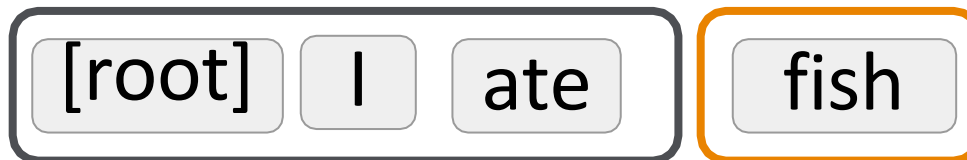
Start



Shift



Shift



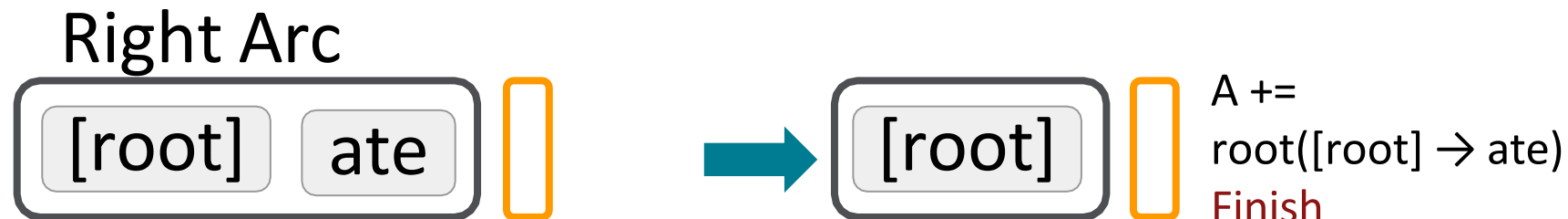
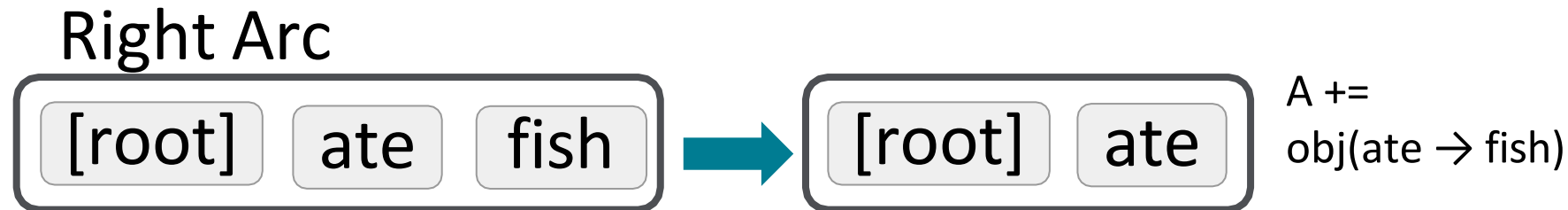
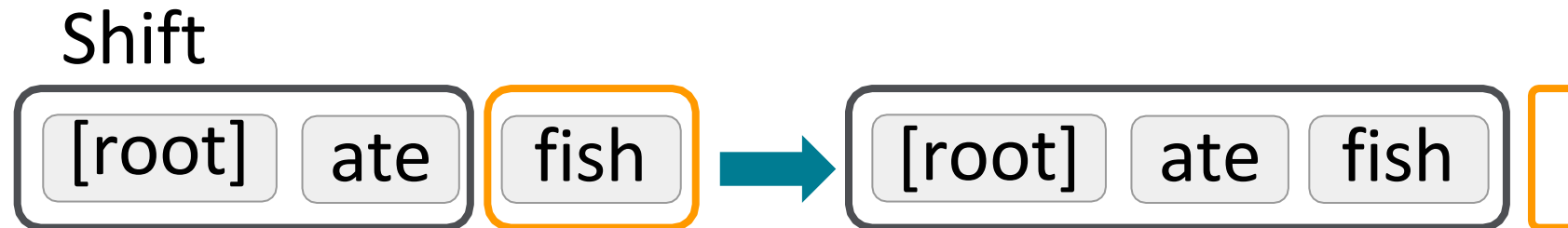
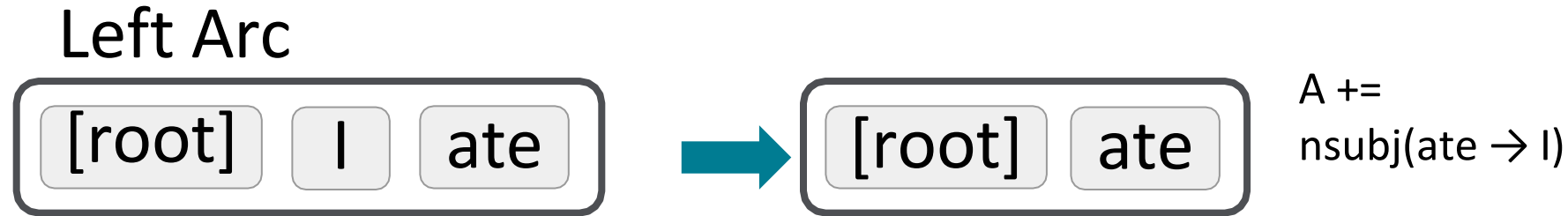
Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$

1. Shift $\sigma, w_i | \beta, A \rightarrow \sigma | w_i, \beta, A$
2. Left-Arc_r $\sigma | w_i | w_j, \beta, A \rightarrow \sigma | w_j, \beta, A \cup \{r(w_j, w_i)\}$
3. Right-Arc_r $\sigma | w_i | w_j, \beta, A \rightarrow \sigma | w_i, \beta, A \cup \{r(w_i, w_j)\}$

Finish: $\sigma = [w]$, $\beta = \emptyset$

Arc-standard transition-based parser

Analysis of “I ate fish”



A = { nsubj(ate → I),
obj(ate → fish)
root([root] → ate) }

Nota bene:

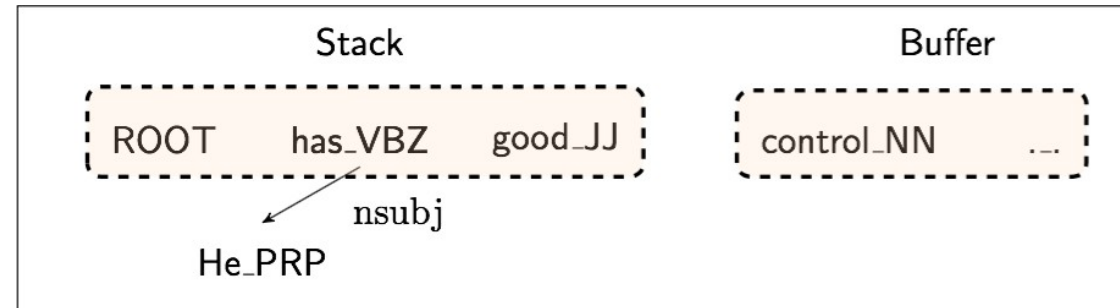
In this example I've at each step made the “correct” next transition.

But a parser has to work this out – by exploring or inferring!

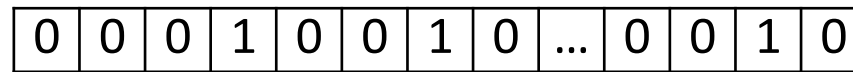
MaltParser [Nivre and Hall 2005]

- We have left to explain how we choose the next action 🙋
 - Answer: **Stand back, I know machine learning!**
- Each action is predicted by a discriminative classifier (e.g., softmax classifier) over each legal move
 - Max of 3 untyped choices (max of $|R| \times 2 + 1$ when typed)
 - Features: top of stack word, POS; first in buffer word, POS; etc.
- There is NO search (in the simplest form)
 - But you can profitably do a beam search if you wish (slower but better):
 - You keep k good parse prefixes at each time step
- The model's accuracy is *a bit* below the state of the art in dependency parsing, but
- It provides **very fast linear time parsing**, with high accuracy – great for parsing the web

Conventional Feature Representation



binary, sparse
dim = $10^6 - 10^7$

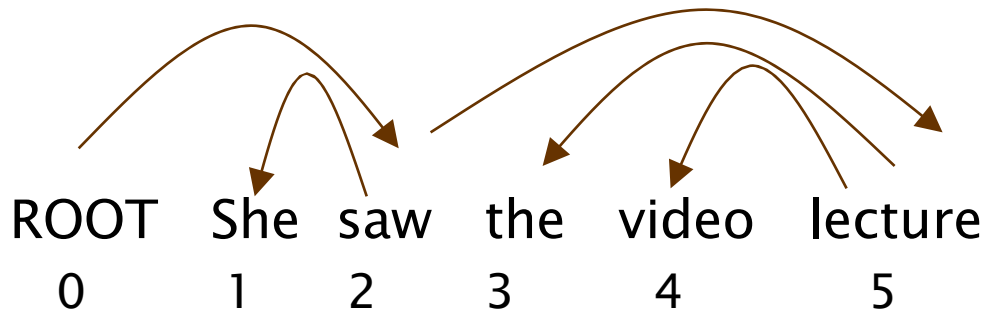


Feature templates: usually a combination of 1–3 elements from the configuration

Indicator features

$s1.w = \text{good} \wedge s1.t = \text{JJ}$
 $s2.w = \text{has} \wedge s2.t = \text{VBZ} \wedge s1.w = \text{good}$
 $lc(s_2).t = \text{PRP} \wedge s_2.t = \text{VBZ} \wedge s_1.t = \text{JJ}$
 $lc(s_2).w = \text{He} \wedge lc(s_2).l = \text{nsubj} \wedge s_2.w = \text{has}$

Evaluation of Dependency Parsing: (labeled) dependency accuracy



$$\text{Acc} = \frac{\text{\# correct deps}}{\text{\# of deps}}$$

$$\text{UAS} = 4 / 5 = 80\%$$

$$\text{LAS} = 2 / 5 = 40\%$$

Gold

1	2	She	nsubj
2	0	saw	root
3	5	the	det
4	5	video	nn
5	2	lecture	obj

Parsed

1	2	She	nsubj
2	0	saw	root
3	4	the	det
4	5	video	nsubj
5	2	lecture	ccomp

Handling non-projectivity

- The arc-standard algorithm we just presented only builds projective dependency trees
- Possible directions to head:
 1. Just declare defeat on nonprojective arcs 🙄
 2. Use dependency formalism which only has projective representations
 - A CFG only allows projective structures; you promote head of projectivity violations
 3. Use a postprocessor to a projective dependency parsing algorithm to identify and resolve nonprojective links
 4. Add extra transitions that can model at least most non-projective structures (e.g., add an extra SWAP transition will allow any non-projectivity, cf. bubble sort)
 5. Move to a parsing mechanism that does not use or require any constraints on projectivity (e.g., the graph-based MSTParser or Dozat and Manning (2017))

4. Why do we gain from a neural dependency parser?

Indicator Features Revisited

Categorical features are:

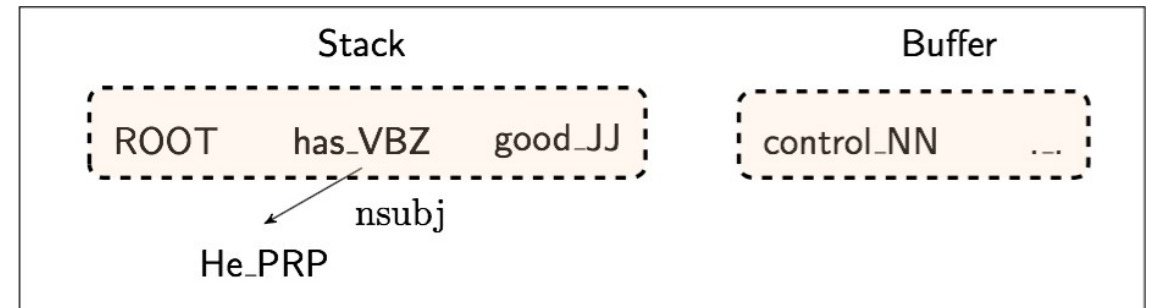
- **Problem #1:** sparse
- **Problem #2:** incomplete
- **Problem #3:** expensive to compute

More than 95% of parsing time is consumed by feature computation

$s1.w = \text{good} \wedge s1.t = \text{JJ}$
 $s2.w = \text{has} \wedge s2.t = \text{VBZ} \wedge s1.w = \text{good}$
 $lc(s_2).t = \text{PRP} \wedge s_2.t = \text{VBZ} \wedge s_1.t = \text{JJ}$
 $lc(s_2).w = \text{He} \wedge lc(s_2).l = \text{nsubj} \wedge s_2.w = \text{has}$

Neural Approach:

learn a dense and compact feature representation



dense
dim = ~1000

0.1 0.9 -0.2 0.3 ... -0.1 -0.5

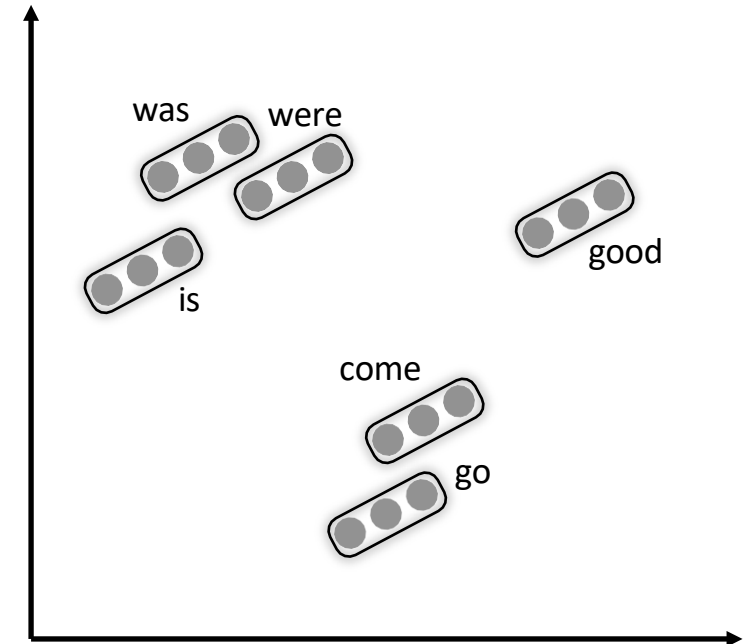
A neural dependency parser [Chen and Manning 2014]

- Results on English parsing to Stanford Dependencies:
 - Unlabeled attachment score (UAS) = head
 - Labeled attachment score (LAS) = head and label

Parser	UAS	LAS	sent. / s
MaltParser	89.8	87.2	469
MSTParser	91.4	88.1	10
TurboParser	92.3	89.6	8
C & M 2014	92.0	89.7	654

First win: Distributed Representations

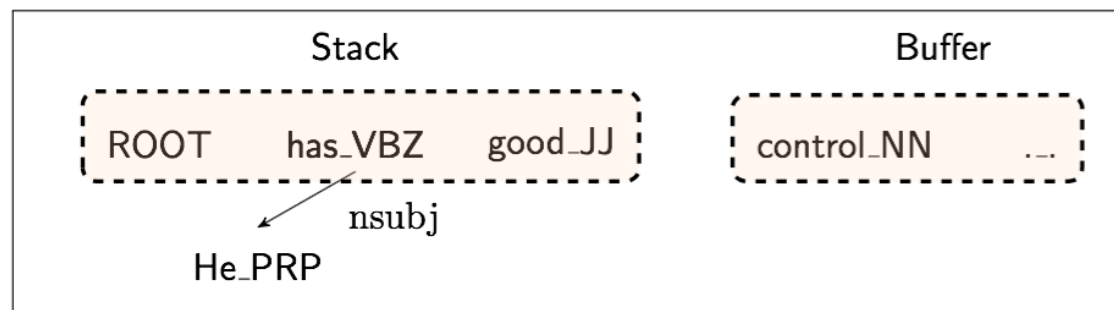
- We represent each word as a d -dimensional dense vector (i.e., word embedding)
 - Similar words are expected to have close vectors.
- Meanwhile, **part-of-speech tags** (POS) and **dependency labels** are also represented as d -dimensional vectors.
 - The smaller discrete sets also exhibit many semantical similarities.



NNS (plural noun) should be close to NN (singular noun).
nummod (numerical modifier) should be close to amod (adjective modifier).

Extracting Tokens & vector representations from configuration

- We extract a set of tokens based on the stack / buffer positions:



	word	POS	dep.
s1	good	JJ	∅
s2	has	VBZ	∅
b1	control	NN	∅
lc(s1)	∅	+	+
rc(s1)	∅	∅	∅
lc(s2)	He	PRP	nsubj
rc(s2)	∅	∅	∅

A concatenation of the vector representation of all these is the neural representation of a configuration

Neural Dependency Parser Model Architecture

(A simple feed-forward neural network multi-class classifier)

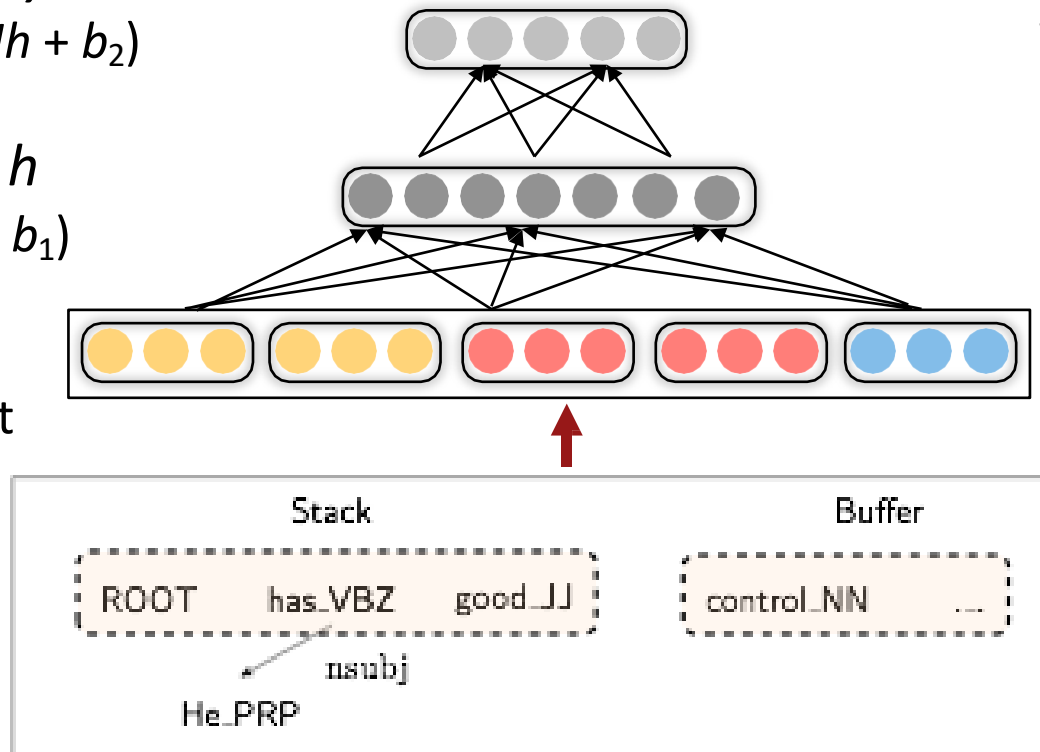
Log loss (cross-entropy error) will be back-propagated to the embeddings

Softmax probabilities → { Shift , Left-Arc_r , Right-Arc_r }

Output layer y
 $y = \text{softmax}(Uh + b_2)$

Hidden layer h
 $h = \text{ReLU}(Wx + b_1)$

Input layer x
lookup + concat

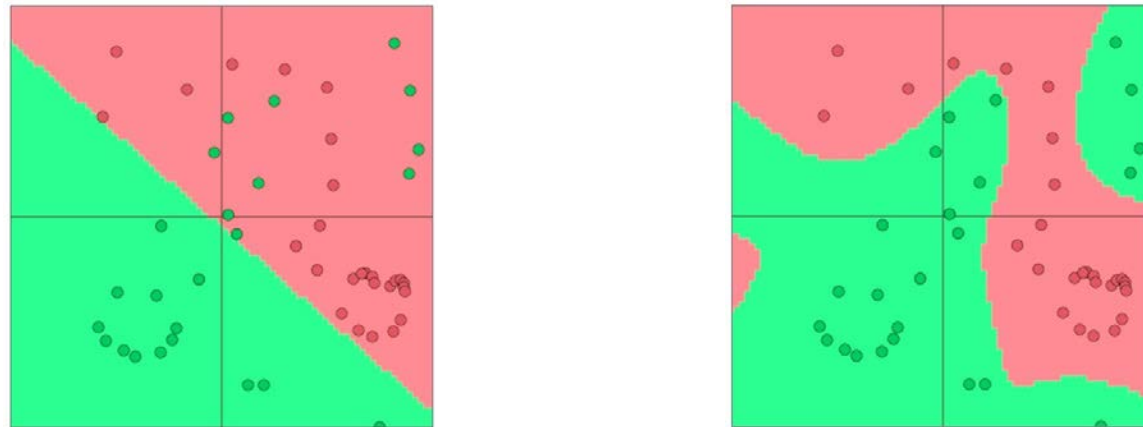


The hidden layer re-represents the input — it moves inputs around in an intermediate layer vector space—so it can be easily classified with a (linear) softmax

Wins:
Distributed representations!
Non-linear classifier!

Second win: Deep Learning classifiers are non-linear classifiers

- Traditional ML classifiers (including Naïve Bayes, SVMs, logistic regression and softmax classifier) are not very powerful classifiers: they only give linear decision boundaries
- But neural networks can use multiple layers to learn much more complex nonlinear decision boundaries



Further developments in transition-based neural dependency parsing

This work was further developed and improved by others, including in particular at Google

- Bigger, deeper networks with better tuned hyperparameters
- Beam search
- Global, conditional random field (CRF)-style inference over the decision sequence

Leading to SyntaxNet and the Parsey McParseFace model (2016):

“The World’s Most Accurate Parser”

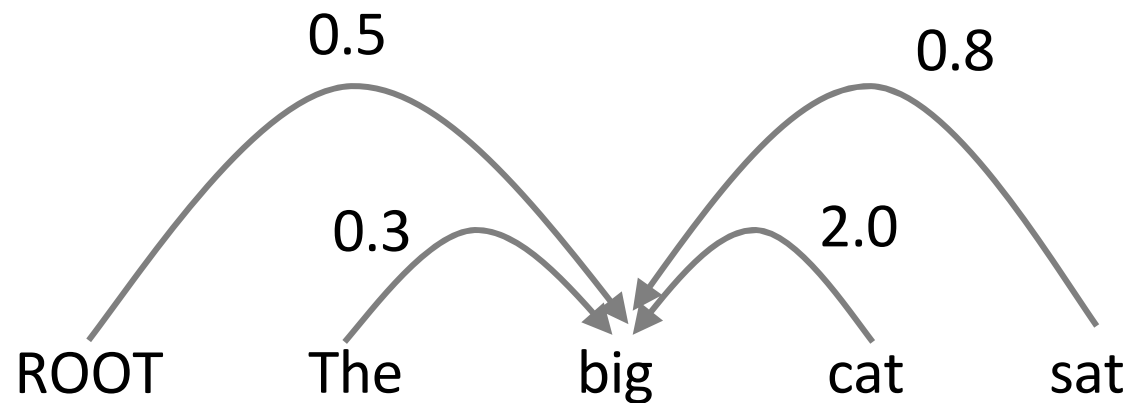
<https://research.googleblog.com/2016/05/announcing-syntaxnet-worlds-most.html>



Method	UAS	LAS (PTB WSJ SD 3.3)
Chen & Manning 2014	92.0	89.7
Weiss et al. 2015	93.99	92.05
Andor et al. 2016	94.61	92.79

Graph-based dependency parsers

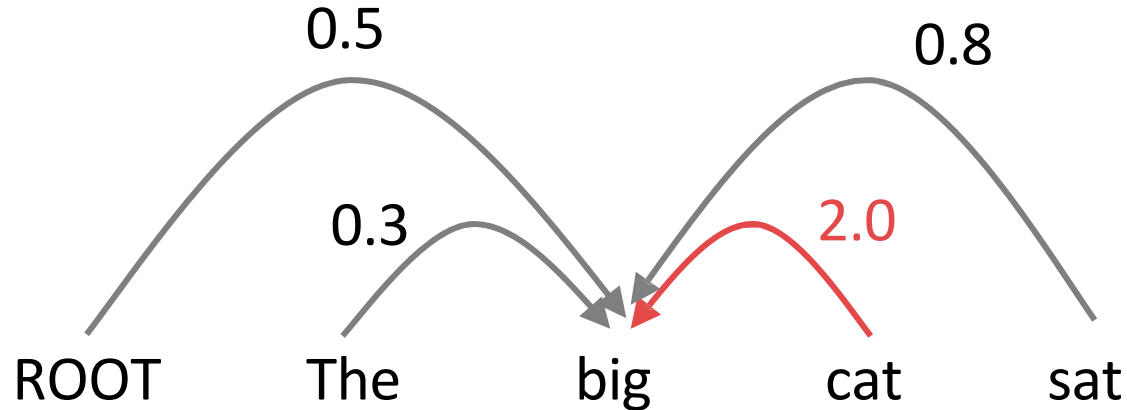
- Compute a score for every possible dependency for each word
 - Doing this well requires good “contextual” representations of each word token, which we will develop in coming lectures



e.g., picking the head for “big”

Graph-based dependency parsers

- Compute a score for every possible dependency (choice of head) for each word
 - Doing this well requires more than just knowing the two words
 - We need good “contextual” representations of each word token, which we will develop in the coming lectures
- Repeat the same process for each other word; find the best parse (MST algorithm)



e.g., picking the head for “big”

A Neural graph-based dependency parser

[Dozat and Manning 2017; Dozat, Qi, and Manning 2017]

- This paper revived interest in graph-based dependency parsing in a neural world
 - Designed a biaffine scoring model for neural dependency parsing
 - Also crucially uses a neural sequence model, something we discuss next week
- Really great results!
 - **But slower than the simple neural transition-based parsers**
 - There are n^2 possible dependencies in a sentence of length n



Method	UAS	LAS (PTB WSJ SD 3.3)
Chen & Manning 2014	92.0	89.7
Weiss et al. 2015	93.99	92.05
Andor et al. 2016	94.61	92.79
Dozat & Manning 2017	95.74	94.08